UNIP – UNIVERSIDADE PAULISTA

Curso de Ciência da Computação

ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

COMPARAÇÃO DE DESEMPENHO E ANÁLISE DE PERFORMANCE DE ALGORITMOS DE ORDENAÇÃO DE DADOS

Emily Fernandes Rodrigues de Souza - RA – F361390

Fernando Henrique de Sá Ferreira - RA - R857449

Francisco dos Santos Dutra - RA – R860825

Gustavo Gonçalves Araujo - RA – F3631C-6

Pedro Lucas Cartagena Silva dos Santos - RA – R669FC2

Leonardo Peres Carvalho de Lima - RA – H032415

Vinicius Andrade Silva Lima - RA – R829910

Nicolas Faustino Reis - RA - R8673C7

São José dos Campos, 02 de Novembro de 2026.

UNIP – UNIVERSIDADE PAULISTA

Curso de Ciência da Computação

ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

COMPARAÇÃO DE DESEMPENHO E ANÁLISE DE PERFORMANCE DE ALGORITMOS DE ORDENAÇÃO DE DADOS

Atividades Práticas Supervisionadas do 3º e 4º Semestre do Curso de Ciência da Computação da **Universidade Paulista – UNIP**.

Coordenador: Prof. Fernando A. **Gotti**

São José dos Campos, 02 de Novembro 2026

UNIP – UNIVERSIDADE PAULISTA

Curso de Ciência da Computação

FICHA DE APROVAÇÃO

TEMA: COMPARAÇÃO DE DESEMPENHO E ANÁLISE DE PERFORMANCE DE ALGORITMOS DE ORDENAÇÃO DE DADOS

Este Trabalho foi aprovado como avaliação semestral da disciplina Atividades Práticas Supervisionadas - APS

Os alunos receberam as seguintes notas:

Aluno	Trabalho Impresso	Apresentação	Média (Total)

Professor Orientador

_____

Stefano B. B. R. . Mathias

São José dos Campos,_____ de _____de 2026.

RESUMO

O presente trabalho, desenvolvido no âmbito das Atividades Práticas Supervisionadas (APS) do curso de Ciência da Computação da Universidade Paulista (UNIP), tem como objetivo analisar e comparar o desempenho de três algoritmos de ordenação: Bubble Sort, Binary Insertion Sort e Bucket Sort, considerando suas características de funcionamento, complexidades computacionais e aplicações no processamento de dados. A pesquisa fundamenta-se em uma abordagem teórica e experimental, buscando relacionar os conceitos estudados na disciplina de Estruturas de Dados com a implementação prática de diferentes métodos de ordenação. A metodologia adotada compreende o levantamento bibliográfico, a implementação dos algoritmos e a realização de testes comparativos utilizando conjuntos de dados numéricos desordenados, compostos por 1.000, 5.000 e 10.000 elementos. Para assegurar maior consistência à análise, foram realizadas sete execuções de cada algoritmo por cenário, considerando-se a média dos tempos de ordenação e desconsiderando-se o período destinado à leitura dos arquivos. Os resultados apresentados indicam que os três algoritmos concluíram satisfatoriamente a ordenação dos conjuntos avaliados, com tempos de execução inferiores a um segundo nas condições experimentais estabelecidas. A análise evidencia, ainda, que as diferentes estratégias de ordenação apresentam particularidades que influenciam sua eficiência computacional, especialmente em relação à quantidade de operações realizadas e às características dos dados processados. Conclui-se que a escolha de um algoritmo de ordenação deve considerar não apenas sua capacidade de organizar corretamente os elementos, mas também fatores relacionados à complexidade computacional, ao volume de informações e às condições de aplicação. Dessa forma, o estudo contribui para a compreensão dos fundamentos da análise de algoritmos e reforça a importância da avaliação de desempenho no desenvolvimento de soluções computacionais eficientes.

Palavras-chave: Algoritmos de ordenação; Estruturas de dados; Bubble Sort; Binary Insertion Sort; Bucket Sort.

ABSTRACT

This study, developed within the scope of the Supervised Practical Activities (APS) of the Computer Science program at the Universidade Paulista (UNIP), aims to analyze and compare the performance of three sorting algorithms: Bubble Sort, Binary Insertion Sort, and Bucket Sort, considering their operating characteristics, computational complexities, and applications in data processing. The research is based on a theoretical and experimental approach, seeking to relate the concepts studied in the Data Structures course to the practical implementation of different sorting methods. The adopted methodology includes a bibliographic review, algorithm implementation, and comparative tests using unordered numerical datasets containing 1,000, 5,000, and 10,000 elements. To ensure greater consistency in the analysis, seven executions of each algorithm were performed for each scenario, considering the average sorting time and excluding the time spent reading the input files. The results indicate that all three algorithms successfully sorted the evaluated datasets, with execution times below one second under the established experimental conditions. The analysis also demonstrates that the different sorting strategies present particular characteristics that influence their computational efficiency, especially regarding the number of operations performed and the characteristics of the processed data. It is concluded that the choice of a sorting algorithm should consider not only its ability to correctly organize the elements but also factors related to computational complexity, data volume, and application conditions. Therefore, this study contributes to the understanding of algorithm analysis fundamentals and reinforces the importance of performance evaluation in the development of efficient computational solutions.

Keywords: Sorting algorithms; Data structures; Bubble Sort; Binary Insertion Sort; Bucket Sort.

LISTA DE TABELAS

Tabela 1 – Exemplo resumido ……. **38**

Tabela 2 – Comparação Bubble Sort 1.000 números …… **53**

Tabela 3 – Comparação Bubble Sort 5.000 números …… **54**

Tabela 4 – Comparação Bubble Sort 10.000 números ……. **54**

Tabela 5 – Comparação Bucket Sort 1.000 números …… **55**

Tabela 6 – Comparação Bucket Sort 5.000 números …… **56**

Tabela 7 – Comparação Bucket Sort 10.000 números ……. **56**

Tabela 8 – Comparação Binary Insertion Sort 1.000 números …….. **57**

Tabela 9 – Comparação Binary Insertion Sort 5.000 números …… **58**

Tabela 10 – Comparação Binary Insertion Sort 10.000 números …….. **58**

LISTA DE FIGURAS

Figura 1 – Bubble Sort Example ……..…… **24**

Figura 2 – Código Bubble Sort ……..… **26**

Figura 3 – Inserindo elementos nos buckets …….. **28**

Figura 4 – Reunindo elementos dos buckets …….. **28**

Figura 5 - Soma do total de comparações …….. **34**

Figura 6 - Número total de deslocamentos ……. **34**

Figura 7 – Definição da estrutura dos nós …….. **38**

Figura 8 – Criação do novo nó …….. **38**

Figura 9 – Inserção no início da lista ……... **39**

Figura 10 – Localização da posição de inserção ……... **39**

Figura 11 – Identificação dos limites do vetor ……. **40**

Figura 12 – Alocação dinâmica dos baldes ……..… **41**

Figura 13 – Distribuição dos elementos entre os baldes …….. **41**

Figura 14 – Reunião dos elementos no vetor original ……... **42**

Figura 15 – Impressão dos elementos do vetor ……. **43**

Figura 16 – Função principal do programa ……. **44**

Figura 17 – Saída do código no terminal ……. **44**

Figura 18 – Criação da função de ordenação ……. **46**

Figura 19 – Laço externo para varrer o vetor várias vezes …….. **46**

Figura 20 – Laço interno para limitar até onde o vetor já está ordenado …….. **46**

Figura 21 – Lógica de permuta (swap) **47**

Figura 22 – Validação se houve alguma troca **47**

Figura 23 – Código principal **48**

LISTA DE ABREVIATURAS E SIGLAS

APS	*Atividades Práticas Supervisionadas*
I/O	*Input/Output (Entrada e Saída)*
POO	*Programação Orientada a Objetos*
RAM	*Random Access Memory (Memória de Acesso Aleatório)*
SSD	*Solid-State Drive (Unidade de Estado Sólido)*
RA	*Registro Acadêmico*

SUMÁRIO

1 - INTRODUÇÃO..... 12

1.1 Contextualização Histórica e Tecnológica.....13

1.2 Apresentação do Problema e dos Algoritmos.....16

1.3 Objetivos e Metodologia da APS..... 19

1.4 Estrutura da APS..... 20

2- REFERENCIAL TEÓRICO..... 23

2.1 Algoritmos de ordenação..... 23

2.2 Bubble Sort..... 24

2.3 BucketSort.....27

2.4 Binary Insertion Sort.....29

3 - DESENVOLVIMENTO.....32

3.1 Binary Insertion Sort.....32

3.1.1 Definição e Conceituação do Binary Insertion Sort..... 32

3.1.2 Mecanismo de Funcionamento e Pseudocódigo.....33

3.1.3 Análise de Complexidade de Tempo e Espaço..... 33

3.1.3.1 Complexidade do Número de Comparações..... 33

3.1.3.2 Complexidade de Deslocamentos e Complexidade Temporal Total..
……34

3.1.3.3 Complexidade Espacial e Estabilidade..... 35

3.1.4 Comparativo com o Insertion Sort Tradicional e Aplicações Práticas.... 35

3.2 Algoritmo Bucket Sort..... 36

3.2.3 Vantagens e Limitações.....38

3.2.4 Definição da estrutura de dados.....39

3.2.11 Análise da complexidade computacional.....46

3.3.2 Funcionamento e Complexidade do Bubble Sort..... 50

4 - RELACIONAMENTO ENTRE DISCIPLINAS..... 52

5 - RESULTADOS E DISCUSSÃO..... 55

6 - CONSIDERAÇÕES FINAIS.....63

7 - REFERÊNCIAS BIBLIOGRÁFICAS.....65

APÊNDICE..... 70

1 - INTRODUÇÃO

Na era contemporânea, o crescimento exponencial do volume de dados gerados diariamente impõe desafios significativos aos sistemas computacionais. A capacidade de armazenar, organizar e recuperar informações de maneira eficiente tornou-se um pilar fundamental para o desenvolvimento de software de alto desempenho (CORMEN, T. H. *et al. Algoritmos: Teoria e Prática, 2024*). Neste contexto, a Ciência da Computação dedica atenção especial ao estudo de estruturas de dados e aos problemas clássicos associados à sua manipulação. Entre estes, a ordenação de dados destaca-se como uma operação basilar, visto que atua como pré-requisito vital para otimizar processos de busca e indexação em praticamente qualquer aplicação tecnológica (ZIVIANI, N. *Projeto de Algoritmos: Com implementações em Pascal e C, 2004*).

Apesar de o objetivo final da ordenação ser universal (organizar um conjunto de elementos segundo uma regra de precedência), os métodos empregues para o alcançar variam drasticamente em termos de lógica matemática e complexidade computacional (SEDGEWICK, R.; WAYNE, K. *Algorithms*, 2011). A escolha do algoritmo ideal não é trivial, dependendo intimamente da natureza dos dados, do volume de processamento exigido e da arquitetura de hardware subjacente. É perante este panorama que se torna imprescindível para o futuro bacharel não apenas conhecimento teórico dos algoritmos, mas também a sua validação empírica através da aplicação prática (TENENBAUM, A. M. *Estruturas de Dados Usando C, 1995*).

O presente estudo concentra-se na análise comparativa de três algoritmos de ordenação com características estruturais e teóricas distintas: o **BubbleSort**, o **BinaryInsertionSort** e o **BucketSort**. Enquanto o BubbleSort representa uma abordagem elementar baseada em trocas adjacentes (frequentemente ineficiente para grandes volumes), o BinaryInsertionSort introduz otimizações lógicas através de buscas binárias para a inserção de elementos (KNUTH, D. E. *The Art of Computer Programming, Volume 3: Sorting and Searching*, 1998). Por sua vez, o BucketSort rompe com o paradigma das comparações diretas, adotando uma abordagem de distribuição dos elementos em recipientes intermediários. A

exploração em conjunto destas três metodologias permite um contraste claro entre algoritmos de diferentes classes de eficiência (CORMEN, T. H. *et al. Algoritmos: Teoria e Prática, 2024*).

Este trabalho é desenvolvido no âmbito das Atividades Práticas Supervisionadas vinculadas à disciplina de Estrutura de Dados. O projeto visa consolidar a aprendizagem através da interdisciplinaridade, promovendo a integração entre os conceitos teóricos lecionados durante o semestre e a sua aplicação prática na resolução de problemas concretos. Para tal, o grupo recebeu a incumbência de implementar os três algoritmos supracitados e submetê-los a testes rigorosos de ordenação. O ambiente de testes utilizará três ficheiros em formato de texto, previamente fornecidos pelo docente, que contêm diferentes conjuntos de dados para simular cenários variados de carga operacional (PLANO DE ENSINO / MANUAL DE ATIVIDADES PRÁTICAS SUPERVISIONADAS (APS). Curso de Ciência da Computação, UNIP, 2026).

Para garantir a integridade da análise de performance e a justa comparação de desempenho entre os algoritmos, estabeleceu-se uma métrica de avaliação estrita. Conforme estipulado nas diretrizes do projeto, a unidade de medida central e exclusiva será o tempo total de ordenação executado pelo processador. É imperativo ressaltar que o tempo de aquisição e leitura dos dados a partir dos ficheiros de texto para a memória não será contabilizado em nenhum dos testes. Este isolamento da métrica garante que os gargalos periféricos de entrada e saída (I/O) não mascaram a verdadeira eficiência do processo específico de ordenação, permitindo conclusões empíricas precisas sobre a viabilidade prática de cada método estudado (PLANO DE ENSINO / MANUAL DE ATIVIDADES PRÁTICAS SUPERVISIONADAS (APS). Curso de Ciência da Computação, UNIP, 2026).

1.1 Contextualização Histórica e Tecnológica

O desenvolvimento dos algoritmos de ordenação está diretamente relacionado à necessidade histórica de organizar e processar grandes quantidades de informações de maneira sistemática. À medida que os sistemas computacionais passaram a manipular volumes cada vez maiores de dados, tornou-se necessário

desenvolver métodos capazes de reorganizar essas informações de acordo com determinados critérios, permitindo que operações posteriores, como pesquisa, indexação e processamento, fossem realizadas de maneira mais eficiente. Nesse contexto, a ordenação tornou-se um dos problemas fundamentais estudados pela Ciência da Computação e pelas disciplinas relacionadas às estruturas de dados (CORMEN et al., 2022).

A importância tecnológica dos algoritmos de ordenação está relacionada não apenas à capacidade de organizar elementos, mas também à influência que a escolha de determinado método exerce sobre o desempenho de um sistema computacional. A quantidade de comparações, trocas, deslocamentos e memória auxiliar utilizada pode variar significativamente entre diferentes algoritmos. Dessa maneira, dois algoritmos capazes de produzir exatamente o mesmo resultado podem apresentar tempos de execução muito diferentes quando submetidos a grandes volumes de dados. O estudo de algoritmos, portanto, envolve tanto a compreensão de sua lógica quanto a análise de sua eficiência computacional (CORMEN et al., 2022; SEDGEWICK; WAYNE, 2011).

Historicamente, os primeiros métodos de ordenação utilizados no desenvolvimento de programas apresentavam estruturas relativamente simples, baseadas principalmente na comparação e movimentação dos elementos. Entre esses métodos encontram-se algoritmos elementares como o Insertion Sort e o Bubble Sort, que continuam sendo utilizados no ensino de estruturas de dados por apresentarem mecanismos de funcionamento relativamente fáceis de compreender e implementar. Apesar de suas limitações de desempenho para grandes volumes de dados, esses algoritmos possuem importância didática por permitirem a introdução de conceitos fundamentais relacionados à complexidade, comparação, troca e organização de elementos (KNUTH, 1998).

Com o avanço da capacidade computacional e o crescimento da quantidade de dados processados, novas estratégias de ordenação foram desenvolvidas com o objetivo de reduzir o custo computacional das operações. Uma dessas estratégias consiste em utilizar informações adicionais sobre os próprios dados para determinar como eles podem ser organizados. Diferentemente dos algoritmos baseados exclusivamente na comparação entre elementos, métodos de distribuição, como o

BucketSort, utilizam o domínio dos valores para distribuí-los em estruturas intermediárias, permitindo que diferentes grupos de elementos sejam processados separadamente. Essa característica possibilita, sob determinadas condições de distribuição dos dados, alcançar desempenho próximo ao linear (CORMEN et al., 2022).

Dentro desse processo de evolução também surgiram variações de algoritmos clássicos destinados a reduzir determinadas operações consideradas custosas. O BinaryInsertionSort constitui uma modificação do Insertion Sort tradicional na qual a busca pela posição adequada de inserção deixa de ser realizada de maneira linear e passa a utilizar uma busca binária. Como a parte já processada do vetor permanece ordenada, é possível utilizar essa propriedade para localizar a posição de inserção de maneira mais eficiente. Entretanto, a redução no número de comparações não elimina o custo dos deslocamentos necessários para inserir o elemento na posição encontrada, mantendo a complexidade geral do algoritmo na ordem de ($O(n^2)$) (KNUTH, 1998).

No cenário tecnológico atual, o estudo comparativo dessas diferentes estratégias permanece relevante porque a eficiência de um algoritmo depende das características do problema e dos dados que serão processados. O estudo de algoritmos de ordenação permite compreender que não existe uma única estratégia adequada para todas as situações, sendo necessário considerar fatores como quantidade de elementos, distribuição dos valores, organização inicial dos dados, memória disponível e operações realizadas durante o processamento. A análise de diferentes técnicas de ordenação constitui, portanto, uma ferramenta importante para o desenvolvimento da capacidade de avaliação e tomada de decisões técnicas na área de Ciência da Computação (CORMEN et al., 2022).

Nesse contexto, o presente trabalho concentra sua análise em três estratégias distintas de ordenação: BubbleSort, BinaryInsertionSort e BucketSort. A escolha desses algoritmos possibilita observar diferentes formas de resolução do mesmo problema computacional, permitindo comparar uma técnica baseada em trocas de elementos adjacentes, uma técnica baseada em inserção com busca binária é uma técnica fundamentada na distribuição dos dados em recipientes

intermediários. Essa diversidade permite estabelecer uma análise teórica e prática das diferenças existentes entre as abordagens estudadas.

1.2 Apresentação do Problema e dos Algoritmos

O problema abordado neste trabalho consiste na ordenação de conjuntos de dados desorganizados, de modo que seus elementos sejam apresentados segundo uma ordem previamente estabelecida. A ordenação constitui uma operação fundamental no processamento de dados, pois estruturas organizadas podem facilitar operações posteriores, como pesquisas, agrupamentos e indexações. De acordo com Cormen et al. (2022), algoritmos de ordenação representam uma parte fundamental do estudo de algoritmos, envolvendo diferentes estratégias para reorganizar uma sequência de elementos de acordo com suas chaves de ordenação.

Para solucionar o problema proposto, serão estudados e implementados três algoritmos de ordenação: BubbleSort, BinaryInsertionSort e BucketSort. Embora os três tenham como finalidade produzir uma sequência ordenada, suas estratégias de funcionamento são distintas. Essa diferença permite analisar como a estrutura interna de cada algoritmo influencia seu comportamento computacional e seu tempo de execução diante de diferentes conjuntos de dados.

O primeiro algoritmo analisado é o BubbleSort, uma técnica de ordenação baseada na comparação de elementos adjacentes. Durante sua execução, elementos vizinhos são comparados e, caso estejam em uma ordem inadequada, suas posições são trocadas. A repetição dessas operações faz com que os maiores elementos sejam progressivamente deslocados para o final da sequência, formando uma região ordenada ao longo das iterações. Uma implementação otimizada pode interromper a execução quando uma passagem completa não produzir nenhuma troca, característica que permite um melhor desempenho quando os dados já estão ordenados ou próximos da ordenação (HARVARD UNIVERSITY, 2026).

Apesar da simplicidade de implementação, o BubbleSort apresenta elevado custo computacional para conjuntos de dados maiores. Em sua forma otimizada, seu melhor caso pode apresentar comportamento linear, (O(n)), quando os elementos já estão ordenados e nenhuma troca é necessária. Entretanto, nos casos médio e pior, seu comportamento permanece quadrático, ($O(n^2)$), devido à quantidade de comparações e trocas realizadas ao longo das passagens (CORMEN et al., 2022; HARVARD UNIVERSITY, 2026). Por essa razão, seu estudo é especialmente relevante para compreender a relação entre simplicidade algorítmica e eficiência computacional.

O segundo algoritmo estudado é o BinaryInsertionSort, uma variação do método InsertionSort. No InsertionSort tradicional, cada novo elemento é comparado com os elementos da parte já ordenada até que seja encontrada sua posição correta. No BinaryInsertionSort, a parte ordenada do vetor é utilizada como base para uma busca binária, reduzindo o número de comparações necessárias para localizar a posição onde o novo elemento deverá ser inserido. Após a identificação dessa posição, os elementos posteriores precisam ser deslocados para liberar o espaço necessário para a inserção (THORNTON, 2022).

A principal diferença entre o InsertionSort convencional e o BinaryInsertionSort está, portanto, na estratégia utilizada para localizar a posição de inserção. A busca binária reduz a quantidade de comparações necessárias para encontrar essa posição, fazendo com que essa etapa apresente comportamento logarítmico. Entretanto, os deslocamentos dos elementos continuam sendo necessários e podem exigir até (O(n)) operações para cada inserção. Dessa forma, embora o BinaryInsertionSort possa apresentar vantagem quando o custo das comparações é significativo, sua complexidade temporal geral permanece ($O(n^2)$) nos casos em que os deslocamentos dominam a execução (THORNTON, 2022).

O terceiro algoritmo analisado é o BucketSort, que apresenta uma abordagem diferente dos dois métodos anteriores. Em vez de realizar exclusivamente comparações entre os elementos da sequência, o algoritmo distribui os valores em diferentes estruturas intermediárias denominadas buckets, ou baldes. Cada elemento é direcionado para determinado bucket de acordo com seu valor ou faixa de valores. Posteriormente, os elementos armazenados em cada bucket são

ordenados individualmente e, ao final, os conteúdos dos buckets são reunidos para formar a sequência ordenada (CORMEN et al., 2022).

O funcionamento do BucketSort está diretamente relacionado à distribuição dos valores de entrada. Quando os elementos estão distribuídos de maneira aproximadamente uniforme entre os buckets, cada estrutura intermediária recebe uma quantidade relativamente pequena de elementos, possibilitando uma ordenação eficiente. Nessa situação, o algoritmo pode apresentar comportamento médio próximo de $(O(n))$, considerando determinadas condições sobre a quantidade de buckets e a distribuição dos dados. Entretanto, quando muitos elementos são concentrados em um único bucket, o desempenho pode ser prejudicado, podendo alcançar $(O(n^2))$ quando um algoritmo como o InsertionSort é utilizado para ordenar os elementos internos (PROGRAMIZ, 2026).

A utilização conjunta dos três algoritmos possibilita estabelecer uma comparação entre estratégias de ordenação com características distintas. O BubbleSort utiliza comparações e trocas entre elementos adjacentes; o BinaryInsertionSort combina a manutenção de uma parte ordenada com a utilização de busca binária para localizar posições de inserção; e o BucketSort utiliza uma estratégia de distribuição dos elementos antes da etapa de ordenação. Dessa forma, o problema proposto não se limita à obtenção de uma sequência ordenada, mas envolve também a investigação do comportamento computacional de diferentes estratégias para solucionar o mesmo problema.

A comparação será realizada considerando conjuntos de dados de diferentes características e tamanhos, conforme estabelecido na metodologia da APS. O desempenho será avaliado a partir do tempo de ordenação, desconsiderando o período necessário para leitura dos arquivos e carregamento dos dados na memória. Essa delimitação é importante para que a medição represente especificamente o custo computacional relacionado à execução dos algoritmos de ordenação, permitindo uma comparação mais controlada entre as três técnicas.

Assim, a investigação proposta busca relacionar os fundamentos teóricos da complexidade de algoritmos com resultados obtidos por meio da execução prática. A análise permitirá verificar de que maneira o comportamento esperado teoricamente

para cada algoritmo se manifesta diante dos conjuntos de dados utilizados no experimento. O estudo dos três métodos contribui, dessa forma, para a compreensão dos impactos que as diferentes estratégias algorítmicas podem produzir sobre o desempenho de programas computacionais.

1.3 Objetivos e Metodologia da APS

O presente projeto de investigação insere-se no contexto das Atividades Práticas Supervisionadas (APS), assumindo-se como um instrumento pedagógico basilar para o aprimoramento da aprendizagem no curso de Ciência da Computação (UNIP, 2026). O objetivo geral destas atividades transcende a mera execução técnica, visando promover uma profunda integração entre a teoria e a prática através da interdisciplinaridade (UNIP, 2026). Pretende-se, deste modo, desenvolver as competências multidisciplinares exigidas aos futuros bacharéis, fomentando não apenas a capacidade de trabalhar em equipa, mas também a responsabilidade técnica e analítica perante problemas complexos de software.

No que concerne aos objetivos específicos, o trabalho foca-se na disciplina de Estruturas de Dados e no problema clássico do armazenamento, ordenação e recuperação de informações (CORMEN et al., 2012). A meta técnica estabelecida para a equipa consiste na implementação, na análise empírica e na comparação rigorosa de desempenho de três algoritmos de ordenação distintos: BubbleSort, BinaryInsertionSort e BucketSort (UNIP, 2026; CORMEN et al., 2012). O intuito é que os alunos compreendam as noções de complexidade computacional inerentes a cada método, traduzindo o conhecimento abstrato obtido em sala de aula para uma linguagem de programação adequada e aplicada a cenários práticos (UNIP, 2026).

A metodologia delineada para sustentar esta avaliação empírica baseia-se na execução de uma bateria de testes controlados. Para o efeito, serão utilizados três ficheiros independentes, em formato de texto, previamente fornecidos pelo docente da disciplina (UNIP, 2026). Estes ficheiros contêm os conjuntos de dados desordenados que atuarão como a carga de processamento exigida para testar os

limites e o comportamento assintótico de cada um dos algoritmos implementados (CORMEN et al., 2012). Adicionalmente, a metodologia inclui uma componente de exposição pedagógica, visto que o docente destacou um destes algoritmos para que o grupo o disse e explique detalhadamente aos restantes alunos da classe (UNIP, 2026).

O pilar central desta metodologia reside no estabelecimento de uma métrica de avaliação de performance estritamente padronizada. Conforme ditam as especificações do projeto, a unidade de medida exclusiva para efeitos de comparação será o tempo total de ordenação despendido pelo processador (UNIP, 2026). É imposta uma premissa fundamental: o tempo de aquisição de dados, ou seja, o intervalo despendido pelo sistema para ler as informações contidas nos ficheiros de texto e carregá-las para a memória principal, não deve, sob qualquer pretexto, ser contabilizado na medição (UNIP, 2026). Esta restrição garante o isolamento da performance, evitando que constrangimentos físicos de leitura de disco (Input/Output) mascaram a verdadeira eficiência do processo específico de ordenação. O ciclo metodológico encerra-se com a validação do trabalho desenvolvido, através da postagem do projeto no portal institucional, onde o professor da disciplina vinculada atesta a conformidade dos resultados com o manual das APS (UNIP, 2026).

1.4 Estrutura da APS

Para assegurar uma exposição lógica, fluida e academicamente rigorosa da investigação conduzida, este documento foi minuciosamente segmentado, respeitando a paginação e as secções exigidas pelo modelo de avaliação. A estrutura do relatório reflete o percurso desde a fundamentação abstrata até à prova empírica dos resultados:

- Referencial Teórico: Esta seção dedica-se à fundamentação bibliográfica do projeto. Servirá para expor os conceitos intrínsecos de complexidade computacional (notação Big-O) e para detalhar a mecânica de funcionamento teórico, os piores casos e os casos

médios de cada um dos três algoritmos de ordenação sob escrutínio: BubbleSort, BinaryInsertionSort e BucketSort.

- Desenvolvimento: Constitui o núcleo técnico e a seção mais extensa do trabalho. Aqui será descrita a arquitetura de software e de hardware utilizada nos testes. Este capítulo apresentará os blocos de código implementados, a técnica de isolamento do tempo de I/O exigida pela métrica e, sobretudo, dedica uma parcela significativa à explicação aprofundada e visual do algoritmo destacado pelo docente para apresentação à turma.

- Relacionamento entre Disciplinas: Um capítulo reservado para a exploração da interdisciplinaridade, evidenciando como os conceitos programáticos de Estruturas de Dados se cruzam em sinergia com outras unidades curriculares do curso de Ciência da Computação para viabilizar a criação deste ambiente de testes.

- Resultados e Discussão: O espaço destinado à apresentação da análise empírica. Através da exposição de gráficos e tabelas, proceder-se-á ao cruzamento e à comparação direta de desempenho, revelando o tempo de execução que o BubbleSort, o BinaryInsertionSort e o BucketSort obtiveram ao processar os diferentes volumes de dados dos três ficheiros de texto. Discutir-se-á se o comportamento real correspondeu à expectativa teórica abordada no Capítulo 2.

- Considerações Finais: Uma síntese analítica onde a equipa expõe as suas conclusões finais sobre as vantagens e as limitações de cada abordagem de ordenação analisada, validando o cumprimento dos objetivos iniciais do projeto.

- Referências Bibliográficas e Apêndice: O fecho do documento garante a validade científica suportando-se nas obras citadas e num apêndice

fundamental para a auditoria do trabalho, o qual incluirá os códigos-fonte produzidos e os registos totais de saída das execuções.

2- REFERENCIAL TEÓRICO

Segundo um artigo publicado pela Wikibooks (2018), “Ordenação é o ato de se colocar os elementos de uma sequência de informações, ou dados, em uma relação de ordem predefinida. O termo técnico em inglês para ordenação é *sorting*, cuja tradução literal é classificação.” Na programação temos muitos momentos onde precisamos lidar com a manipulação de dados, e isso pode ser facilitado usando essa ordenação, para tal organização temos uma sequência dada, e seu problema de ordenação e uma permutação desta sequência, tal que vamos do 1° termo até o termo não desejados. É interessante quando vemos que algumas ordenações são facilmente definidas como, a ordem numérica, ou a ordem alfabética, crescentes ou decrescentes, Mas quando essa escala fica maior ou exige maior complexidade, a que entra os algoritmos de ordenação (Wikibooks, 2018).

2.1 Algoritmos de ordenação

Algoritmos de ordenação são sequências finitas e definidas de instruções na qual irão reorganizar os elementos de uma coleção, seguindo uma ordem de comparação, que pode ser crescente ou decrescente. Muito importante para o meio da ciência computacional, já que praticamente todo software atual implementa dados ordenados em algum ponto seja em seus, bancos de dados que indexam registros,sistemas de procura, e-commerces, planilhas, entre outras. E sem essa ordenação, operações como busca binária, junções de tabela e renderização de listas tornam-se inviáveis em escala (DOUGLAS, 2026).

Um artigo da Shiftmind publicado por Henry Douglas (2026) explica como um algoritmo de ordenação funciona. Segundo Henry, todo algoritmo de ordenação opera em uma estrutura de dados linear e executa uma função de comparação entre os pares de elementos, sendo o resultado desta comparação o que vai determinar se há ou não uma troca de posições. As diversas variações entre esses algoritmos surgem de uma estratégia que visa diminuir o número total de operações necessárias.

E elas podem se diferenciar também entre, algoritmos comparativos e distributivos. Os comparativos como (Bubble Sort, Quick Sort, Merge Sort, Heap Sort) usam operadores relacionais (<, >, =) para decidir a ordem dos elementos. O limite teórico inferior desses algoritmos é O(n log n), provado matematicamente por árvores de decisão. Por outro lado, os algoritmos de distribuição (Radix Sort, Counting Sort, Bucket Sort) não fazem comparação de elementos diretamente, eles distribuem valores em baldes ou processam dígitos separadamente, sendo sua complexidade linear O(n+k) sob certas restrições de domínio dos dados (DOUGLAS, 2026).

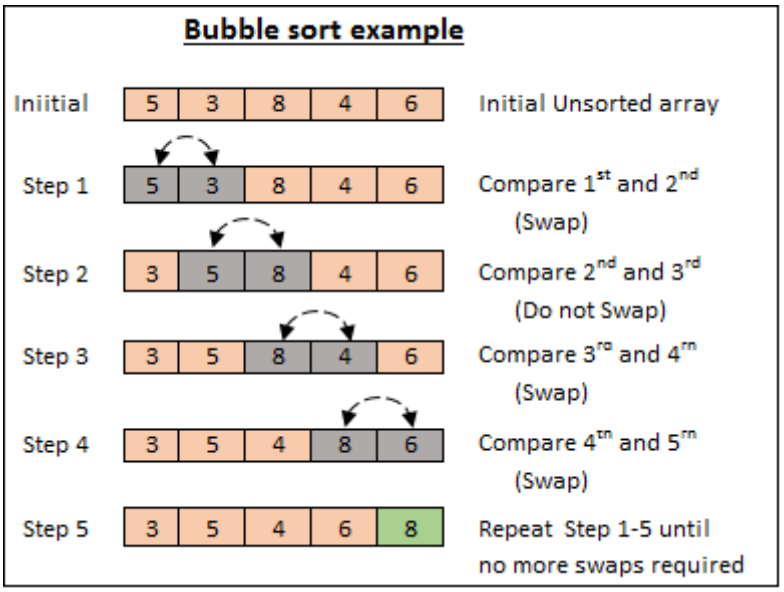
Temos também a divisão desses algoritmos em in-place e out-of-place. O in-place ordena a coleção sem alocar memória adicional significativa, usando apenas O(1) ou O(log n) de espaço auxiliar. Já os algoritmos out-of-place precisam de uma estrutura auxiliar com o tamanho da entrada, e exige O(n) de memória extra. Em cenários onde há restrição de RAM como, sistemas embarcados, processamento de arquivos gigantes, a escolha entre in-place e out-of-place é essencial. (DOUGLAS, 2026).

Com base em tudo que vimos, agora sabemos que a escolha do algoritmo certo para o problema ideal, impacta diretamente a performance da aplicação, pois ordenar 10 milhões de linhas com Bubble Sort pode levar horas, enquanto Quicksort e Merge Sort resolvem o mesmo problema em segundos. Conhecer os tipos, complexidade computacional, o famoso Big O e os trade-offs entre tempo, memória e estabilidade são essenciais para qualquer desenvolvedor que trabalha com volumes reais de dados (DOUGLAS, 2026).

2.2 Bubble Sort

“O Bubble Sort é um algoritmo de classificação comumente usado em ciência da computação” (BLOG DA TRYBE, 2022). Ele se baseia na ideia de comparar repetidamente os pares de elementos adjacentes e, logo após, os troca de posições se os mesmos estiverem na ordem errada, como mostrado na figura 1.

Figura 1 – Bubble Sort Example



Fonte : Googleusercontent, 2022.

O algoritmo de classificação por bolha, é uma das classificações mais famosas, e da própria classificação em bolha existem milhares de mutações a partir dela, e são usadas principalmente para fins educacionais.

E para que usamos essa classificação em bolha, é bom usar esse tipo de classificação em bolha para organizar uma sequência de números em uma certa ordem. Este é o tipo mais simples de classificação, ela não costuma ser muito usada pela ciência da computação do mundo real, seus principais usos para programadores são, Como forma de aprendizado para ensinar programadores novos e programadoras a classificar conjuntos de dados, já que é um algoritmo simples e fácil de entender e implementar. Também pode ser usado como metodologia para classificar pequenos conjuntos de dados, pois como passar por cada número adjacente pode ser um trabalho demorado quando usamos um conjunto de dados muito grande, já com um pequeno número de elementos pode funcionar melhor. E

por fim podemos usar em um conjunto de dados, onde os mesmos já estão em maioria na ordem, pois alguns cientistas da computação e analistas de dados, usam esse algoritmo como uma verificação final para conjuntos de dados no qual eles acreditam já estarem em ordem quase classificada (BLOG DA TRYBE, 2022).

E suas complexidades são descritas pela CoddyTech (2026) como no seu melhor caso temos a complexidade: O(n) já ordenado, com uma verificação de saída antecipada, no caso médio temos, $O(n^2)$ em uma ordem aleatória, e por fim no seu pior caso temos, $O(n^2)$ onde todos os dados estão ordenados ao contrário.

Para entender o Bubble Sort de uma forma interessante e divertida podemos usar o próprio aplicativo da CoddyTech, onde podemos inserir números, e testarmos todos os casos usando Bubble Sort, no aplicativo usam-se o código mostrado na figura 2.

Figura 2 – Código Bubble Sort

```
static void bubblesort(int tamanho,int []vetor){
int temp = 0;
for (int i= 0;i<tamanho;i++){
    for(int j=1;j<tamanho-i;j++)
        if(vetor[j-1] > vetor [j]){
            //trocar os elementos
            temp = vetor[j-1];
            vetor[j-1] = vetor[j];
            vetor[j] = temp;
        }
}
}
```

Fonte: Ana Borowsky, 2025.

2.3 BucketSort

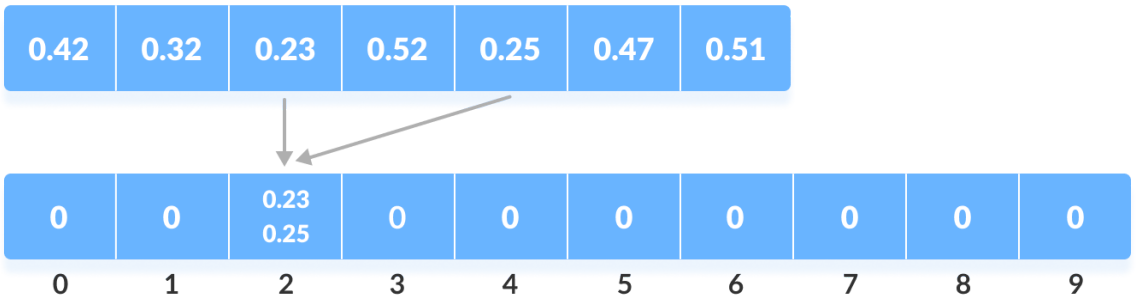
O BucketSort é uma técnica de ordenação na qual o algoritmo divide elementos em diversos grupos, ou buckets (baldes). Esses buckets são formados pela distribuição uniforme dos elementos. E uma vez que esses elementos são divididos em buckets, eles podem ser ordenados usando qualquer outro tipo de algoritmo de ordenação, e no final de tudo, os elementos ordenados são reunidos de forma ordenada. Eles podem funcionar bem quando os elementos da array de entrada estão distribuídos uniformemente por um intervalo, ele é um algoritmo muito estável, pois usamos Insertion Sort (que é estável) para ordenar os buckets individuais (Geeksforgeeks, 2025).

E o que algoritmos estáveis quer dizer. Bem de acordo com a (Wikipedia, 2025), algoritmos estáveis são aqueles que preservam a ordem original dos registros que possuem o mesmo valor de chave, Ou seja, se dois elementos possuem valores iguais na lista de entrada, eles aparecerão na mesma ordem na lista de saída, extremamente importante quando existem dados associado que precisam ser mantidos em relação à ordem original.

E na prática, como esse algoritmo funciona, pela descrição passo a passo da Programiz (2026), primeiro vamos imaginar uma array, na qual possui 6 elementos indo do índice 0 até o indicie 5, sendo compostos pelos números 0.42, 0.32, 0.23, 0.52, 0.25, 0.47, 0.51 nas suas respectivas ordem.

Agora devemos Criar um array de tamanho 10, onde cada slot deste array é usado como um balde para armazenar elementos, depois insira elementos nos baldes a partir do array criado, esses elementos serão inseridos de acordo com o alcance do balde. No código de exemplo, temos baldes de cada um onde se variam de 0 a 1, 1 a 1, 2 a 3,.....(n-1) para n. Suponhamos que um desses elementos de entrada seja tomado como o 0.23, ele será multiplicado pelo tamanho do array, que nesse exemplo é igual a 10, depois obtemos o resultado 2.3 e igualamos ele a um inteiro, onde obteremos o número 2 e aí sim jogamos esse número no seu respectivo índice, ou seja, o valor inteiro é igual a 2, ele é jogado na array no índice 2. E da mesma forma fazemos isso com todos os números como mostrado na figura 3.

Figura 3 – Inserindo elementos nos buckets

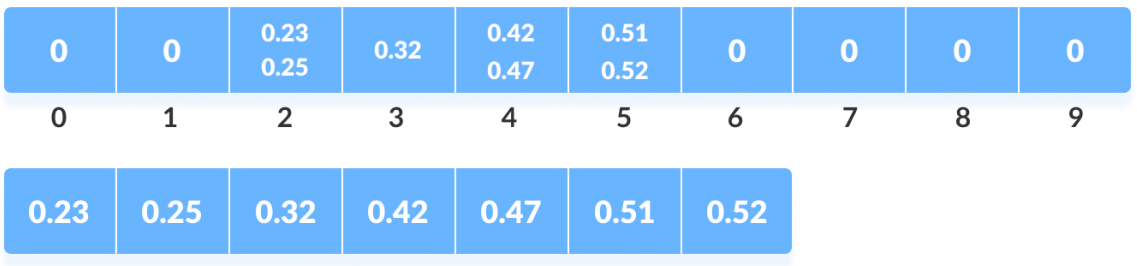


Fonte: Programiz, 2026.

Por fim, os elementos de cada balde são ordenados usando qualquer um dos tipos algoritmos de ordenação estável. Os elementos de cada balde são coletados,

Isso é feito iterando pelo balde e inserindo um elemento individual no array original em cada ciclo. O elemento do balde é apagado assim que é copiado para o arranjo original, assim tendo nosso conjunto de dados certamente ordenados como mostrado nas figuras 4.

Figura 4 – Reunindo elementos dos buckets



Fonte: Programiz, 2026.

A inúmeras vantagens em usar o algoritmo de ordenação BucketSort, entre eles. A eficiência, pois dividindo o conjunto de dados em grupos menores, temos

uma redução significativa no número de comparações necessárias, resultando em um tempo de execução mais rápido, especialmente em conjuntos grandes e esparsos. Outra grande vantagem ao usar o BucketSort é a adaptabilidade, já que ele pode ser otimizado para diferentes tipos e distribuições de dados, podendo se ajustar facilmente para lidar com dados numéricos, sequências de texto ou outros tipos de dados, o que o deixa extremamente versátil. Por fim também temos a paralelização, já que em sua natureza de dividir para conquistar, o BucketSort é altamente paralelizável, o que significa que podemos aproveitar ao máximo os sistemas de computação distribuídas e processadores multi core para obter um desempenho ainda melhor (TecnoDigital, 2025).

Sua complexidade se dá no pior caso: $O(n^2)$ acontece quando um balde recebe todos os elementos, nesse caso, estaremos executando a ordenação por inserção em todos os itens, o que fará com que a complexidade de tempo seja $O(n^2)$. Podemos reduzir a complexidade de tempo do pior caso para O(n log n) usando um algoritmo O(n log n) como Merge Sort ou Heap Sort para ordenar os buckets individuais, mas podemos deixar ainda melhor para casos onde os buckets possuem um número pequeno de itens, pois a inserção funciona melhor em arrays menores. No melhor caso temos O(n + k) que ocorre quando cada balde recebe o mesmo número de elementos, neste caso, todas as chamadas de ordenação por inserção levam tempo constante, já que o número de itens em cada bucket será constante.

2.4 Binary Insertion Sort

A ordenação binária ou como também chamada por ordenação por inserção binária, é uma variação de um outro algoritmo chamado Insertion Sort, sendo que a única diferença é que, em vez de escanear a parte ordenada linearmente para achar a posição correta de inserção, nos usamos a busca binária, tornando a busca mais rápida (Algorithmroom, 2026).

E como esse algoritmo de ordenação funciona, como dito no artigo da geeksforgeeks (2025), neste algoritmo dividimos os mesmos membros em dois

subarrays, sendo eles o filtrado e o não filtrado. O primeiro elemento dos mesmos membros está no subarray organizado, sendo que todos os outros não são planejados. Em seguida, iteramos do segundo elemento ao último, na repetição do i-ésimo, tornamos o objeto atual nossa “chave”, sendo a chave um recurso no qual devemos adicionar à nossa lista existente abaixo. Para que isso seja possível, primeiro usaremos uma busca binária no subarray ordenado abaixo para localizar um elemento maior que nossa chave, chamamos essa posição de poço e depois deslocamos todos os elementos para a direita, em seguida para 1 e criamos Array[pos] = chave. Podemos notar que em toda i-ésima multiplicação, a parte esquerda do array até (i - 1) já está ordenada.

E como implementamos essa abordagem bem, ela pode ser implementada de tal modo:

> Altere o array do segundo elemento até o último;
> Armazene o elemento atual A[i] em uma chave variável;
> Encontre a posição do elemento logo maior que A[i] no subarray de A[0] a A[i-1] usando busca binária. Digamos que esse elemento esteja em posição de índice (point de referência);
> Desloque todos os elementos do ponto índice para o i-1 à direita;
> A[pos] = chave (Geeksforgeeks, 2025).

Para uma Matriz Inicial de {37, 23, 0, 17, 12, 72, 31}, temos um exemplo em forma de tabela como visto abaixo:

Tabela 1 – Exemplo resumido

Passe	Inserido	Pos binário	Ação	Array resultante
1	23	0	Shift 37 → Inserir em 0	{23, 37, 0, 17, 12, 72, 31}
2	0	0	Turno 23, 37 → Inserir em 0	{0, 23, 37, 17, 12, 72, 31}
3	17	1	Turno 23, 37 → Inserir às 13	{0, 17, 23, 37, 12, 72, 31}
4	12	1	Turno 17, 23, 37 → Inserir às 13	{0, 12, 17, 23, 37, 72, 31}
5	72	5	Sem → de turno Inserir às 5	{0, 12, 17, 23, 37, 72, 31}
6	31	4	Turno 37, 72 → Inserir às 16	{0, 12, 17, 23, 31, 37, 72}

Fonte: Algorithmroom, 2025.

E no fim, temos uma complexidade no pior caso de execução igual a $O(n^2)$, devido à série de trocas necessárias para cada inserção (Geeksforgeeks, 2025).

3 - DESENVOLVIMENTO

O estudo de algoritmos de ordenação constitui um dos pilares fundamentais da ciência da computação, uma vez que a organização eficiente de dados otimiza diretamente a recuperação de informações e o desempenho geral de sistemas computacionais. Para a materialização prática dessas lógicas, a linguagem C destaca-se pelo controle refinado de baixo nível e pela capacidade de manipulação direta de estruturas na memória (*in-place*). Desse modo, abordagens com características e custos computacionais distintos — como o *Bubble Sort*, o *Bucket Sort* e o *Binary Insertion Sort* — podem ser implementadas e analisadas de forma a evidenciar suas particularidades operacionais (CORMEN et al., 2012).

3.1 Binary Insertion Sort

O desenvolvimento deste estudo aborda o algoritmo de ordenação **Binary Insertion Sort** (Ordenação por Inserção Binária), analisando sua fundamentação teórica, seu mecanismo interno de execução, sua complexidade computacional de tempo e espaço e suas aplicações práticas no contexto da Ciência da Computação.

3.1.1 Definição e Conceituação do Binary Insertion Sort

O *Binary Insertion Sort* é uma variação otimizada do algoritmo tradicional *Insertion Sort* (Ordenação por Inserção). O *Insertion Sort* clássico opera construindo de forma iterativa uma sublista ordenada na porção esquerda do arranjo e inserindo, a cada passo, o próximo elemento não ordenado na posição correta dessa sublista. Para identificar a posição adequada de inserção, a versão tradicional realiza uma busca linear varrendo os elementos da sublista um a um da direita para a esquerda.

A modificação introduzida pelo *Binary Insertion Sort* consiste na substituição da busca linear pela **busca binária** (*binary search*) para localizar o índice exato onde o elemento selecionado (denominado *chave* ou *key*) deve ser inserido na sub lista já ordenada. Uma vez que o segmento esquerdo do arranjo permanece ordenado a cada iteração, a propriedade de ordenação garante a aplicabilidade da busca binária.

Conforme destacam a literatura e os manuais de algorítmica, o objetivo primário dessa alteração é reduzir a quantidade de comparações executadas entre as chaves do vetor, reduzindo a complexidade de busca por elemento de no pior caso para .

3.1.2 Mecanismo de Funcionamento e Pseudocódigo

O funcionamento do algoritmo fundamenta-se na manutenção de dois sub arranjos conceituais dentro do vetor original A de tamanho n: a) a sub lista ordenada *A 0…i-1*, que se expande progressivamente; b) a sub lista não ordenada *A i…n-1*, da qual os elementos são retirados sequencialmente.

O processo iterativo ocorre do segundo elemento (*i=n*) até o último elemento (*i=n-1*). A execução de cada iteração desdobra-se em três etapas bem definidas:

1. **Seleção da Chave**: O elemento A i é armazenado em uma variável temporária `chave` (ou `selected`).
2. **Busca Binária do Ponto de Inserção**: É realizada uma busca binária no intervalo ordenado *A 0…i-1* para encontrar o índice onde a `chave` deve ser posicionada de modo a preservar a ordem não decrescente.
3. **Deslocamento de Elementos e Inserção**: Todos os elementos localizados entre o índice *pos* e *i-1* são deslocados uma posição para a direita (*A j+1 = A j*), abrindo espaço na posição , onde a `chave` é finalmente depositada (*A pos = chave*).

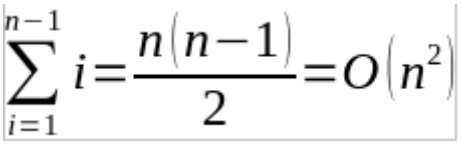
3.1.3 Análise de Complexidade de Tempo e Espaço

A análise assintótica do *Binary Insertion Sort* revela uma importante assimetria entre a complexidade do número de comparações e a complexidade do número de deslocamentos na memória.

3.1.3.1 Complexidade do Número de Comparações

Na busca binária realizada sobre uma sublista de tamanho , o número de comparações necessárias para encontrar a posição *pos* é limitado por $l^2(i)$. Somando o total de comparações ao longo de todas as *n-1* iterações:

Figura 5 - Soma do total de comparações



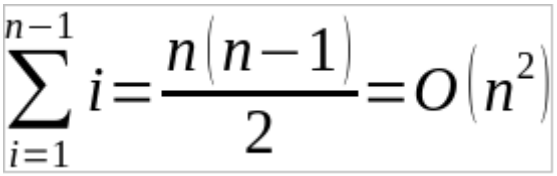
Fonte: The Algorithm Design Manual 3, 2020

Utilizando a aproximação de Stirling, conclui-se que o número de comparações no pior caso e no caso médio é da ordem de *O n|n* . Trata-se de uma melhoria assintótica expressiva em comparação ao *Insertion Sort* tradicional, que requer *O n²* comparações no pior caso.

3.1.3.2 Complexidade de Deslocamentos e Complexidade Temporal Total

Apesar da redução drástica no número de comparações, o algoritmo ainda precisa inserir o elemento na posição correta de uma estrutura de dados de vetor contíguo na memória. No pior caso (quando o vetor está ordenado de forma invertida), cada iteração exige o deslocamento de elementos para a direita. O número total de deslocamentos é dado pela fórmula 1 abaixo:

Figura 6 - Número total de deslocamentos



Fonte: The Algorithm Design Manual 3, 2020

Como o tempo de execução é dominado pelas operações de gravação e movimentação na memória, a complexidade temporal total no pior caso e no caso médio permanece .

No **melhor caso** (quando o vetor já se encontra ordenado), a busca binária ainda executa comparações no total na implementação padrão, enquanto a versão linear tradicional consegue diagnosticar o vetor ordenado em *O n* comparações.

3.1.3.3 Complexidade Espacial e Estabilidade

- **Complexidade Espacial**: O algoritmo é executado *in-place* (no próprio arranjo), exigindo apenas memória auxiliar constante para variáveis de controle como chave, inicio, fim, meio e pos. Portanto, sua complexidade espacial auxiliar é $O|1$.
- **Estabilidade**: O algoritmo é **estável**, desde que a função de busca binária seja projetada para retornar o índice imediatamente posterior ao último elemento igual à chave quando houver chaves duplicadas, preservando assim a ordem relativa original dos elementos com valores equivalentes.

3.1.4 Comparativo com o Insertion Sort Tradicional e Aplicações Práticas

A comparação entre o *Insertion Sort* linear tradicional e o *Binary Insertion Sort* traz lições fundamentais para o projeto e análise de algoritmos:

1. **Custo de Comparação vs. Custo de Movimentação**: O *Binary Insertion Sort* destaca-se em cenários nos quais a operação de comparação de chaves possui alto custo computacional em relação ao deslocamento de dados na memória. Exemplos clássicos incluem a ordenação de vetores de *strings* longas ou de estruturas de objetos complexos com chaves compostas, onde avaliar *A meio* < *chave* envolve processamento computacional elevado.
2. **Limitações em Vetores Contíguos**: Em dados numéricos simples mantidos em arranjos contíguos de memória, o ganho obtido com a busca binária é frequentemente atenuado pelo custo de descolar os elementos no laço interno. Estudos analíticos demonstram que a redução no tempo prático de execução ao substituir a busca linear pela binária em vetores aleatórios limita-se a um teto de aproximadamente 33,3%.
3. **Uso em Algoritmos Híbridos**: O *Binary Insertion Sort* é amplamente empregado na prática como sub-rotina para pequenos subarranjos em algoritmos de ordenação híbridos e de alta performance. Algoritmos como o *Timsort* (padrão em Python e Java) e variantes do *QuickSort* utilizam variações de ordenação por inserção quando as partições se tornam suficientemente pequenas (geralmente $n<=32$ elementos), aproveitando a baixa sobrecarga e o excelente desempenho em pequenos conjuntos de dados.

3.2 Algoritmo Bucket Sort

O Bucket Sort é um algoritmo de ordenação que consiste em distribuir os elementos de um conjunto em diferentes compartimentos, denominados baldes, realizar a ordenação dos elementos dentro desses compartimentos e, posteriormente, reuni-los em uma única sequência ordenada. A implementação desenvolvida em linguagem C utiliza listas encadeadas para armazenar os elementos de cada balde e permite trabalhar com números positivos e negativos.

O funcionamento do algoritmo foi dividido em quatro etapas principais: identificação dos valores mínimo e máximo, inicialização dos baldes, distribuição dos elementos e coleta dos valores ordenados. A seguir, são apresentadas as principais estruturas e funções utilizadas na implementação.

3.2.1 Conceito Bucket Sort

O Bucket Sort funciona por meio da distribuição dos elementos em um conjunto finito de baldes. Cada balde representa uma determinada faixa de valores. Depois da distribuição, os baldes são ordenados individualmente e, por fim, seus conteúdos são percorridos em ordem para formar novamente o vetor de saída. Essa sequência de distribuição, ordenação local e concatenação constitui a estrutura básica do algoritmo(WIKIPÉDIA, 2021; GEEKSFORGEEKS, 2025).

No modelo analisado por Bercea e Even, a entrada possui n chaves no intervalo [0, 1). O balde j é definido como o conjunto das chaves pertencentes ao intervalo [j/n, (j+1)/n). Depois da distribuição, os baldes são ordenados separadamente e a saída é obtida percorrendo-os em ordem crescente. A atribuição inicial das chaves aos baldes e a varredura final dos baldes possuem custo linear, de modo que a análise pode se concentrar no custo da ordenação realizada dentro deles (BERCEA; EVEN).

O material da UFPR apresenta a mesma ideia de forma algorítmica: cria-se um vetor de baldes, cada posição recebe uma lista vazia, cada elemento A[i] é inserido no balde determinado por n·A[i], cada lista é ordenada com Insertion Sort e,

por fim, as listas são concatenadas em ordem. O material também destaca que o Bucket Sort não é uma ordenação por comparação e que a análise apresentada é probabilística, pois o tempo de execução depende da distribuição das entradas (VIGNATTI, s.d.).

3.2.2 Funcionamento o Algoritmo

O funcionamento pode ser dividido em quatro etapas principais. Primeiro, são criados os baldes vazios. Segundo, cada elemento do vetor de entrada é direcionado ao balde correspondente à sua faixa de valores. Terceiro, cada balde não vazio é ordenado individualmente. Quarto, os baldes são percorridos em ordem e seus elementos são colocados novamente no vetor original ou concatenados para produzir a saída ordenada (WIKIPÉDIA, 2021).

No procedimento apresentado pela UFPR, considerando valores no intervalo [0, 1), o índice do balde é obtido por n·A[i]. Assim, valores próximos entre si tendem a ser direcionados para o mesmo balde. O exemplo visual fornecido no material mostra dez elementos sendo distribuídos entre dez baldes; posteriormente, os elementos de cada lista são ordenados e as listas são concatenadas na ordem dos índices dos baldes (VIGNATTI, s.d.).

A implementação apresentada pela GeeksforGeeks utiliza n baldes e, para cada elemento, calcula o índice por meio de n multiplicado pelo valor do elemento, convertido para inteiro. Em seguida, aplica Insertion Sort individualmente aos baldes e concatena os resultados. A mesma página apresenta implementações em C++, Java, Python, C#, JavaScript e uma descrição passo a passo do processo (GEEKSFORGEEKS, 2025).

Exemplos:

Um dos exemplos fornecidos pelas fontes utiliza o vetor [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68]. Para esse exemplo, são utilizados dez baldes. A distribuição apresentada no material da UFPR coloca .12 e .17 no balde 1; .21, .23 e .26 no balde 2; .39 no balde 3; .68 no balde 6; .72 e .78 no balde 7; e .94 no balde 9. Os demais baldes permanecem vazios (VIGNATTI, s.d.).

Depois da distribuição, cada balde é ordenado individualmente. No caso do exemplo, o balde 1 já contém os valores .12 e .17 em ordem; o balde 2 passa a conter .21, .23 e .26; e o balde 7 contém .72 e .78. A concatenação dos baldes, começando pelo índice menor e avançando até o maior, produz a sequência ordenada 0.12, 0.17,0.21, 0.23, 0.26, 0.39, 0.68, 0.72, 0.78 e 0.94 (GEEKSFORGEEKS, 2025; VIGNATTI, s.d.).

O exemplo permite visualizar a principal propriedade estrutural do método: não é necessário ordenar diretamente todos os elementos juntos desde o início. A distribuição por faixas organiza previamente os elementos em grupos e a ordenação é realizada dentro desses grupos. A etapa final apenas percorre os baldes na ordem estabelecida para reconstruir a sequência completa.

3.2.3 Vantagens e Limitações

Entre as características favoráveis apresentadas nas fontes está a possibilidade de obter tempo linear sob uma distribuição uniforme adequada. A UFPR demonstra o tempo esperado Q(n) para o modelo analisado, enquanto a Wikipédia afirma comportamento linear quando os valores são uniformemente distribuídos. A GeeksforGeeks também destaca que o algoritmo funciona bem quando os elementos de entrada estão uniformemente distribuídos em um intervalo.

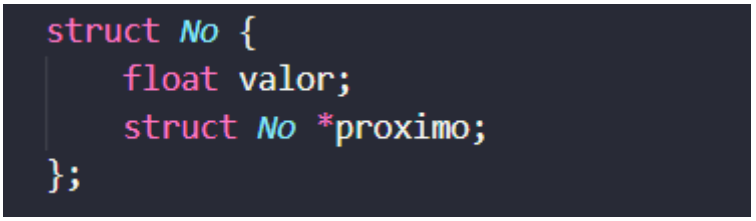
Outra característica é a possibilidade de escolher o método utilizado dentro dos baldes. As fontes apresentam Insertion Sort como uma opção e também mencionam que os baldes podem ser ordenados por outros algoritmos. Bercea e Even estudam explicitamente duas classes, uma baseada em algoritmo quadrático e outra baseada em algoritmo b log b. Dessa forma, o comportamento do Bucket Sort não depende apenas da distribuição, mas também da estratégia de ordenação utilizada em cada balde.

Como limitação, as fontes deixam claro que o bom desempenho depende das características da entrada. A análise linear apresentada não deve ser interpretada como independente da distribuição: ela pressupõe a distribuição uniforme especificada. Além disso, a Wikipédia informa $O(n^2)$ para o pior caso. No exemplo em C disponível nessa fonte, também é observado que aquela implementação não trata elementos negativos sem um tratamento especial (WIKIPÉDIA, 2021).

3.2.4 Definição da estrutura de dados

A primeira etapa da implementação consiste na definição da estrutura responsável por representar os elementos das listas encadeadas utilizadas nos baldes, conforme apresentado no Figura 6.

Figura 6 – Definição da estrutura dos nós



Fonte: Autor (2026).

A estrutura “**struct No**” possui dois campos: “valor”, responsável por armazenar o número, e “proximo”, que contém um ponteiro para o próximo nó da lista. Quando o campo “proximo” recebe o valor NULL, significa que não existe outro elemento após o nó atual, sendo ele assim, o último elemento da lista.

A utilização de listas encadeadas permite armazenar os elementos dinamicamente, sem exigir que cada balde possua uma quantidade fixa de posições previamente estabelecidas. Essa característica possibilita a inserção de novos elementos durante a execução do algoritmo.

3.2.5 Função de inserção ordenada

A função “inserirOrdenado” tem como objetivo realizar a inserção de elementos nos respectivos baldes, garantindo que os valores sejam armazenados em ordem crescente. Para isso, a função recebe dois parâmetros: o endereço do primeiro elemento da lista, representado por “*lista”, e o valor que será inserido, identificado como “valor”. A partir dessas informações, a função verifica a posição adequada para inserir o novo elemento, preservando a ordenação dos valores já existentes. Esse processo pode ser observado na Figura 7.

Figura 7 – Criação do novo nó

```c
struct No *inserirOrdenado(struct No *lista, float valor) {
    struct No *novoNo = (struct No *)malloc(sizeof(struct No));
    novoNo->valor = valor;
    novoNo->proximo = NULL;
```

Fonte: Autor (2026).

Nesse trecho, a função malloc realiza a alocação dinâmica de memória para armazenar um novo nó. Em seguida, o campo “valor” recebe o número a ser inserido, enquanto “proximo” é inicializado com NULL, indicando que o nó ainda não está conectado à lista.

Posteriormente, a função verifica se a lista está vazia ou se o novo elemento deve ocupar a primeira posição, conforme apresentado no Figura 8.

Figura 8 – Inserção no início da lista

```c
if (lista == NULL || lista->valor >= valor) {
    novoNo->proximo = lista;
    return novoNo;
}
```

Fonte: Autor (2026).

Caso a lista esteja vazia ou o novo valor seja menor ou igual ao primeiro elemento, o novo nó passa a ocupar a primeira posição. Para isso, seu ponteiro “proximo” recebe o endereço do antigo início da lista, e a função retorna o endereço do novo nó.

Quando o elemento não deve ocupar a primeira posição, a função percorre a lista até encontrar o local adequado para sua inserção, conforme demonstrado no Figura 9.

Figura 9 – Localização da posição de inserção

```
struct No *atual = lista;
while (atual->proximo != NULL && atual->proximo->valor < valor) {
    atual = atual->proximo;
}

novoNo->proximo = atual->proximo;
atual->proximo = novoNo;

return lista;
```

Fonte: Autor (2026).

O ponteiro atual percorre os nós da lista até encontrar a posição correspondente ao novo valor. Quando essa posição é localizada, os ponteiros são atualizados para inserir o novo nó entre os elementos existentes. Dessa maneira, a lista permanece ordenada após cada inserção, eliminando a necessidade de executar uma ordenação adicional dentro de cada balde.*(explicar com mais detalhes a inserção)*

3.2.6 Identificação dos valores mínimo e máximo

A função principal de ordenação começa verificando se o vetor possui elementos. Em seguida, identifica os valores mínimo e máximo, que serão utilizados como referência para a distribuição dos números entre os baldes, conforme apresentado no Figura 10.

Figura 10 – Identificação dos limites do vetor

```c
if (n <= 0) return;

// 1. Encontra o menor e o maior valor do vetor para fazer a
normalização
float menor = vetor[0];
float maior = vetor[0];
for (int i = 1; i < n; i++) {
    if (vetor[i] < menor) menor = vetor[i];
    if (vetor[i] > maior) maior = vetor[i];
}

// Se todos os elementos forem iguais, já está ordenado
if (menor == maior) return;
```

Fonte: Autor (2026).

Inicialmente, as variáveis menor e maior recebem o primeiro elemento do vetor. O laço de repetição percorre os demais elementos, atualizando essas variáveis sempre que encontra um valor inferior ao mínimo atual ou superior ao máximo atual.

Ao final, a condição “menor == maior” verifica se todos os elementos possuem o mesmo valor. Caso isso ocorra, o vetor já está ordenado e a função pode ser encerrada sem executar as etapas seguintes.

Essa identificação permite adaptar a distribuição dos elementos ao intervalo numérico presente na entrada, possibilitando a utilização de valores positivos e negativos.

3.2.7 Inicialização dos baldes

Após identificar os limites do vetor, o algoritmo cria a estrutura responsável por armazenar os baldes. A implementação utiliza um vetor de ponteiros para nós, inicializado por meio da função calloc, conforme demonstrado no Figura 11.

Figura 11 – Alocação dinâmica dos baldes

```c
struct No **baldes = (struct No **)calloc(n, sizeof(struct No *));
if (baldes == NULL) {
    printf("Erro de alocação de memória.\n");
    return;
}
```

Fonte: Autor (2026).

A variável baldes representa um vetor no qual cada posição armazena o endereço do primeiro nó de uma lista encadeada. A função calloc aloca memória para n ponteiros e inicializa suas posições com zero, representado por NULL nesse contexto. Assim, todos os baldes começam vazios.

A condição seguinte verifica se a alocação foi realizada corretamente. Caso o retorno de calloc seja NULL, uma mensagem de erro é apresentada e a execução da função é encerrada.

3.2.8 Distribuição dos elementos

A distribuição, também denominada scatter, consiste em determinar o balde correspondente a cada elemento do vetor e inserir o valor na lista encadeada correspondente. Essa etapa é apresentada no Figura 12.

Figura 12 – Distribuição dos elementos entre os baldes

```c
for (int i = 0; i < n; i++) {
    // Mapeia o valor linearmente para o intervalo [0, 1) e multiplica por n
    int indiceBalde = (int)((vetor[i] - menor) / (maior - menor) * (n - 1));

    // Insere o elemento ordenadamente no respetivo balde
    baldes[indiceBalde] = inserirOrdenado(baldes[indiceBalde], vetor[i]);
}
```

Fonte: Autor (2026).

O laço de repetição percorre todos os elementos do vetor. Para cada valor, é calculado o índice do balde por meio de uma expressão que considera o elemento atual, os limites mínimo e máximo e a quantidade de elementos da entrada.

A expressão normaliza o valor em relação ao intervalo numérico identificado e o mapeia proporcionalmente para os índices entre 0 e n - 1, sendo n a quantidade

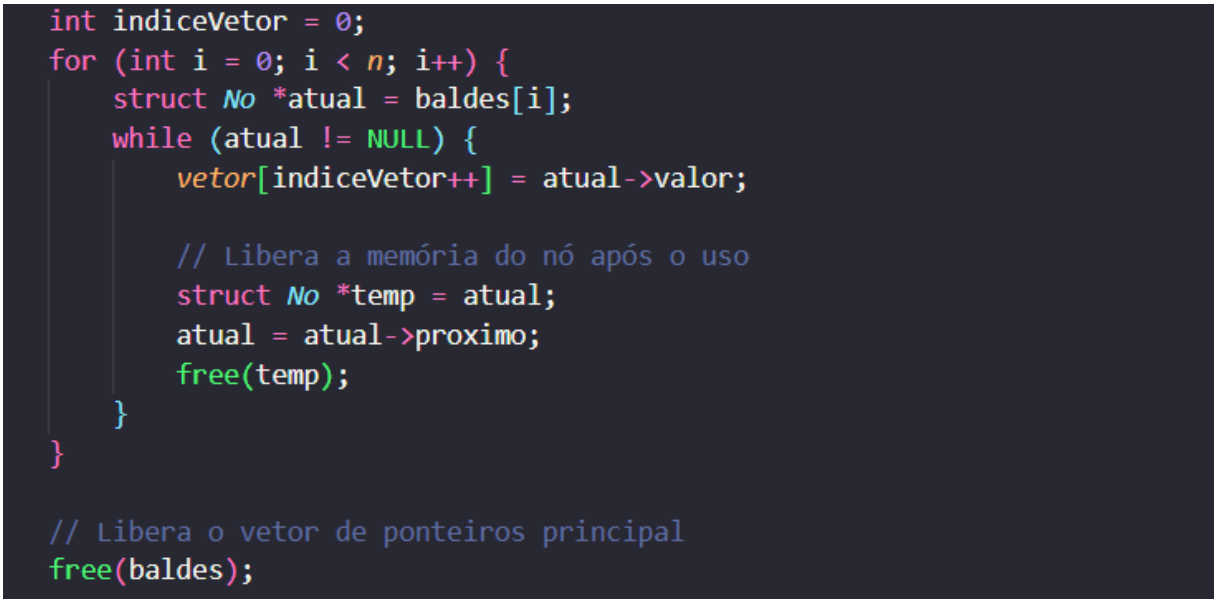
de elementos a serem ordenados. Dessa forma, o menor valor é associado ao primeiro balde e o maior valor ao último, enquanto os demais elementos são distribuídos proporcionalmente entre os índices disponíveis.

Após determinar o índice, a função “inserirOrdenado” é chamada para adicionar o elemento à lista correspondente, mantendo os valores daquele balde em ordem crescente.

3.2.9 Coleta dos elementos ordenados

A coleta, também denominada gather, consiste em percorrer os baldes e transferir seus elementos de volta para o vetor original. Essa operação é apresentada no Figura 13.

Figura 13 – Reunião dos elementos no vetor original



```
int indiceVetor = 0;
for (int i = 0; i < n; i++) {
    struct No *atual = baldes[i];
    while (atual != NULL) {
        vetor[indiceVetor++] = atual->valor;

        // Libera a memória do nó após o uso
        struct No *temp = atual;
        atual = atual->proximo;
        free(temp);
    }
}

// Libera o vetor de ponteiros principal
free(baldes);
```

Fonte: Autor (2026).

A variável “indiceVetor” controla a posição em que cada elemento será armazenado no vetor original. O laço externo percorre os baldes em ordem crescente de índice, enquanto o laço interno percorre os nós de cada lista encadeada.

Como os elementos de cada balde já estão ordenados e os baldes são percorridos sequencialmente, a coleta permite reconstruir o vetor em ordem crescente.

Após copiar o valor de cada nó, o algoritmo armazena temporariamente seu endereço na variável “temp”, avança para o próximo elemento e libera a memória utilizada pelo nó com a função free. Ao final da operação, a memória destinada ao vetor de ponteiros dos baldes também é liberada.

Esse procedimento contribui para o gerenciamento adequado da memória alocada dinamicamente durante a execução do algoritmo.

3.2.10 Função de impressão e execução do programa

A função “imprimirVetor” é utilizada para apresentar os elementos do vetor na tela, permitindo visualizar os resultados antes e depois da ordenação, conforme demonstrado no Figura 14.

Figura 14 – Impressão dos elementos do vetor

```
void imprimirVetor(float vetor[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%.2f ", vetor[i]);
    }
    printf("\n");
}
```

Fonte: Autor (2026).

A função percorre todas as posições do vetor e apresenta seus valores com duas casas decimais, utilizando o especificador de formato “%.2f”. Após a impressão dos elementos, o comando printf("\n") realiza uma quebra de linha.

A execução do programa ocorre por meio da função main, responsável por declarar o vetor de entrada, determinar sua quantidade de elementos e chamar as funções necessárias para a ordenação, conforme apresentado na Figura 15.

Figura 15 – Função principal do programa

```c
int main() {
    // Testando com números fora do intervalo de 0 a 1 (ex: negativos e números maiores)
    float vetor[] = {1511, -18463, -37407, 63983, -10575, -91889, 64628, -77710, 15482, 22387};
    int n = sizeof(vetor) / sizeof(vetor[0]);

    printf("Vetor original: \n");
    imprimirVetor(vetor, n);

    bucketSort(vetor, n);

    printf("\nVetor ordenado: \n");
    imprimirVetor(vetor, n);

    return 0;
}
```

Fonte: Autor (2026).

O vetor inicial contém dez valores inteiros armazenados no tipo float, incluindo números positivos e negativos. A quantidade de elementos é calculada pela divisão do tamanho total do vetor pelo tamanho de um de seus elementos.

Em seguida, o programa apresenta o vetor original, executa a função bucketSort e imprime o resultado da ordenação, conforme pode ser observado na Figura 16. O comando return 0 indica o encerramento normal do programa.

Figura 16 - Saída do código no terminal

```
Problemas  Saída  Console de Depuração  Terminal  Portas

PS C:\Users\gusta\OneDrive\Documentos\Estudos unip> cd "c:\Users\gusta\OneDrive\Documentos\Estudos unip\" ;
Vetor original:
1511.00 -18463.00 -37407.00 63983.00 -10575.00 -91889.00 64628.00 -77710.00 15482.00 22387.00

Vetor ordenado:
-91889.00 -77710.00 -37407.00 -18463.00 -10575.00 1511.00 15482.00 22387.00 63983.00 64628.00
```

Fonte: Autor(2026)

3.2.11 Análise da complexidade computacional

A complexidade computacional do Bucket Sort depende da distribuição dos elementos entre os baldes e do custo das operações realizadas dentro de cada lista encadeada.

Na implementação analisada, a identificação dos valores mínimo e máximo apresenta complexidade temporal (O(n)), assim como a inicialização dos baldes e a

coleta dos elementos. Entretanto, a distribuição pode apresentar custos diferentes, pois a função “inserirOrdenado” percorre os nós existentes para localizar a posição correta de cada novo elemento.

Em condições favoráveis de distribuição, nas quais as listas permanecem pequenas, o algoritmo pode apresentar complexidade temporal esperada de (O(n)). No pior caso, quando muitos elementos são concentrados em um único balde, as inserções podem exigir um número quadrático de operações, resultando em complexidade ($O(n^2)$).

A complexidade espacial é ($O(n)$), uma vez que são utilizados um vetor de baldes proporcional ao tamanho da entrada e nós de listas encadeadas para armazenar os elementos.

Portanto, a implementação demonstra que a eficiência do Bucket Sort está relacionada não apenas à quantidade de elementos processados, mas também à distribuição dos valores e ao procedimento utilizado para ordená-los internamente.

3.2.12 Considerações finais sobre a implementação

A implementação do *Bucket Sort* em linguagem C demonstra a utilização de estruturas de dados dinâmicas para organizar elementos numéricos em ordem crescente. A identificação dos valores mínimo e máximo permite adaptar o mapeamento dos elementos ao intervalo presente no vetor, enquanto as listas encadeadas possibilitam manter os valores de cada balde ordenados durante a distribuição.

Além disso, a liberação da memória dos nós e do vetor de baldes evidencia a importância do gerenciamento de recursos em programas. Entretanto, recomenda-se verificar o retorno da função malloc antes de acessar o novo nó, a fim de evitar acessos inválidos em situações de falha de alocação.

Conclui-se que o algoritmo apresenta uma alternativa de ordenação cuja eficiência depende das características dos dados de entrada e da distribuição dos elementos entre os baldes. Sua análise permite compreender a relação entre estruturas de dados, alocação dinâmica de memória e complexidade computacional em algoritmos de ordenação.

3.3 Algoritmo Bubble Sort

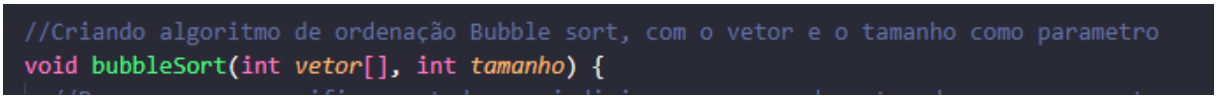
O algoritmo *Bubble Sort* (frequentemente traduzido como ordenação por flutuação ou método da bolha) baseia-se na comparação sequencial entre pares de elementos vizinhos, efetuando a troca de suas posições sempre que o item antecedente apresentar um valor superior ao subsequente (FATEC FRANCA, 2023). Esse ciclo de varredura repete-se em múltiplas iterações sobre o conjunto de dados.

3.3.1 Implementação do código

A implementação computacional do algoritmo Bubble Sort foi desenvolvida na linguagem C, estruturando-se de maneira modular e operando diretamente na memória (in-place), o que elimina a necessidade de alocação de estruturas de dados auxiliares. O funcionamento do código divide-se logicamente nos componentes detalhados a seguir.

Em primeiro lugar, a Figura 17 mostra a função principal de ordenação, denominada bubbleSort, que recebe como parâmetros de entrada o vetor de números inteiros (vetor[]) e a sua respectiva dimensão (tamanho):

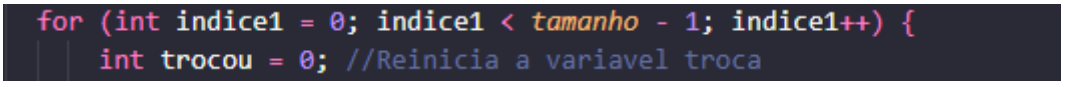
Figura 17 - Criação da função de ordenação



Fonte: Elaborado pelo Autor(2026).

O controle macro das passagens sequenciais é exercido por um laço externo governado pela variável “indice1”, que assegura o número necessário de varreduras para cobrir toda a extensão do arranjo. Além disso, no início de cada ciclo externo, implementou-se um mecanismo de otimização baseado em uma variável de controle booleana chamada “trocou”, reinicializada com o valor 0 (falso), conforme apresentada na Figura 18.

Figura 18 - Laço externo para varrer o vetor várias vezes



Fonte: Elaborado pelo Autor(2026).

O escopo interno, controlado pela variável “indice2”, é responsável por comparar os elementos adjacentes. Conforme percebemos na Figura 19, o limite superior deste laço decresce progressivamente por meio da subtração do índice externo (tamanho - 1 - indice1), impedindo que o algoritmo reexamine posições que já foram consolidadas nas extremidades finais do vetor.

Figura 19 - Laço interno para limitar até onde o vetor já está ordenado

```
for (int indice2 = 0; indice2 < tamanho - 1 - indice1; indice2++) {
```

Fonte: Elaborado pelo Autor(2026).

Quando o algoritmo constata que o elemento atual excede o valor do seu sucessor (vetor[indice2] > vetor[indice2 + 1]), executa-se a lógica de permuta (swap). Uma variável temporária, designada “guardaValor”, armazena transitoriamente o dado para salvaguardar a integridade informacional, permitindo a realocação dos valores em ordem crescente e a atribuição de valor verdadeiro (1) à variável trocou, o que informa ao código que durante o laço, o programa realizou a troca de posição de algum elemento, conforme conferimos na Figura 20.

Figura 20 - Lógica de permuta(swap)

```
if (vetor[indice2] > vetor[indice2 + 1]) {
    int guardaValor = vetor[indice2];
    vetor[indice2] = vetor[indice2 + 1];
    vetor[indice2 + 1] = guardaValor;
    trocou = 1;
}
```

Fonte: Elaborado pelo Autor(2026).

Para garantir o ganho de eficiência no melhor caso, logo após o término do laço interno, a condição “if (!trocou)” avalia se nenhuma alteração ocorreu durante a varredura. Caso verdadeiro, significa que o vetor está ordenado, e o comando “break” é acionado para interromper imediatamente o fluxo de repetição, caracterizando o mecanismo de paragem antecipada (early exit), conforme evidencia a Figura 21.

Figura 21 - Validação se houve alguma troca

```c
if (!trocou) {
    break; //Quebra o laço for, pois não há mais trocas
}
```

Fonte: Elaborado pelo Autor(2026).

Por fim, o procedimento principal (main) encarrega-se da validação prática do algoritmo. Inicializa-se um vetor heterogêneo contendo valores inteiros positivos e negativos, calculando-se dinamicamente a sua dimensão por meio da divisão do espaço total em bytes ocupado pelo arranjo pelo tamanho de um único elemento (sizeof(vetor) / sizeof(vetor[0])). Após a chamada da função de ordenação, os dados organizados são apresentados na saída padrão, conforme apresentado na Figura 22.

Figura 22 - Código principal

```c
int main() {
  //Criando um vetor de 7 números com indicies de 0 a 6
    int vetor[] = {1511, -18463, -37407, 63983, -10575, -91889, 64628, -77710, 15482, -64598};
    //Criando a variavel de tamanho, com uma divisão entre a quantidade total de bytes do vetor pela quantidade total de um elemento do vetor
    int tamanho = sizeof(vetor) / sizeof(vetor[0]);

    //Chamando nosso algoritmo
    bubbleSort(vetor, tamanho);

    //Imprimindo o nosso vetor ordenado
    printf("Vetor ordenado: ");
    //Percorre todo o vetor, imprimindo um número ao lado do outro
    for (int indice1 = 0; indice1 < tamanho; indice1++) {
        printf("%d ", vetor[indice1]);
    }

    //Retorna 0 se tudo der certo
    return 0;
}
```

Fonte: Elaborado pelo Autor(2026).

3.3.2 Funcionamento e Complexidade do *Bubble Sort*

Na implementação prática baseada na linguagem C, o código emprega uma estratégia clássica de otimização por meio de uma variável de controle booleana, tratada como trocou. Através dela, o algoritmo monitora dinamicamente se ocorreu alguma permuta de elementos durante a passagem do laço interno (indice2). Caso o vetor já se encontre ordenado e nenhuma alteração seja efetuada ao longo de uma varredura completa (!trocou), a execução é imediatamente interrompida por meio do comando break, evitando iterações redundantes.

3.3.2.1 Análise de Complexidade Assintótica

A eficiência assintótica do Bubble Sort varia conforme o estado inicial dos dados de entrada. No pior caso e no caso médio, com complexidade $O(n^2)$, o algoritmo lida com dados em ordem decrescente ou aleatória, cenários em que o duplo aninhamento de repetições obriga o sistema a efetuar o número máximo de comparações e permutas. Por outro lado, o melhor caso atinge uma complexidade linear $O(n)$ quando o vetor já se encontra totalmente ordenado; graças à implementação da estrutura de verificação de troca (trocou), o algoritmo executa apenas uma única passagem e encerra a operação antecipadamente, diferenciando-se das versões estáticas tradicionais que mantêm a complexidade quadrática independentemente da ordenação prévia (CORMEN et al., 2012).

4 - RELACIONAMENTO ENTRE DISCIPLINAS

O desenvolvimento do projeto representa a aplicação prática e detalhada das disciplinas de Programação Orientada a Objetos e Estrutura de Dados, ambas desenvolvidas em Java. Ao longo do semestre, o estudo dessas matérias permitiu compreender que a programação vai muito além da escrita dos comandos. Ela é uma forma de organizar o pensamento para resolver problemas complexos e, no nosso caso, foi uma ferramenta fundamental para gerenciar e ordenar dados de forma eficiente. No contexto deste projeto, a POO e a Estrutura de Dados foram os instrumentos que permitiram transformar a lógica abstrata de ordenação em um software funcional, intuitivo e rápido.

Compreender como os dados interagem com as classes em Java nos deu a visão necessária para escolher as melhores soluções de código. Os algoritmos de ordenação trabalhados no trabalho — o Bubble Sort, o Binary Insertion Sort e o Bucket Sort — mostraram na prática como a teoria vista em sala de aula se aplica no dia a dia do desenvolvimento de software.

4.1 A Estruturação do Mundo Real em Objetos

A premissa da disciplina de Programação Orientada a Objetos é a capacidade de enxergar o mundo real e as regras do sistema como um conjunto de entidades que possuem características e comportamentos. Essa visão foi essencial para a estruturação da nossa aplicação, onde os conjuntos de dados deixaram de ser

apenas números soltos e passaram a ser tratados como objetos organizados dentro da memória.

Ao criarmos a lógica para manipular os elementos do sistema, aplicamos o conceito de Abstração. Isso significa que selecionamos apenas o que era importante para a ordenação e para o processamento do programa (como as chaves de comparação e os valores dos vetores), descartando informações irrelevantes para o algoritmo. Essa capacidade de filtrar a realidade e transformá-la em código foi o que permitiu que o programa fosse direto ao ponto e entregasse o resultado esperado com rapidez.

4.2 A Importância da Organização e do Encapsulamento

Um dos pontos mais marcantes da disciplina foi o aprendizado sobre o encapsulamento. Na prática, este conceito funciona como uma caixa de proteção em volta das informações mais importantes do sistema. Na nossa aplicação, isso foi applied para garantir que os dados armazenados nas estruturas não fossem alterados ou corrompidos por falhas externas ou acessos indevidos no meio das execuções dos métodos de ordenação.

Garantir a integridade das listas e vetores antes de rodar os testes de ordenação evitou inconsistências de memória e permitiu medir com precisão o tempo e o comportamento de cada rotina implementada.

4.3 Aplicação Prática das Estruturas de Dados e Algoritmos

Se a POO nos deu os caminhos para organizar o sistema, a matéria de Estrutura de Dados trouxe o motor necessário para fazê-lo rodar com eficiência. A escolha e o estudo dos algoritmos de ordenação foram fundamentais para entender como o computador lida com coleções de elementos na prática:

- **Bubble Sort:** Foi o primeiro algoritmo implementado, funcionando por meio de comparações simples entre elementos vizinhos e trocas sucessivas. Mesmo sendo um método mais lento e com complexidade $O(n^2)$, sua implementação ajudou bastante a entender o funcionamento básico dos laços de repetição e o controle de ponteiros em vetores em Java.

- **Binary Insertion Sort:** Surgiu como uma evolução direta do Insertion Sort tradicional. Ao utilizar a busca binária para encontrar a posição correta onde o elemento deve ser inserido, conseguimos reduzir bastante a quantidade de comparações feitas pelo programa. Isso mostrou na prática como a lógica de busca impacta diretamente o tempo de resposta da aplicação.

- **Bucket Sort:** Foi a solução utilizada para cenários com distribuição mais uniforme de dados. Ao dividir o vetor principal em vários 'baldes' menores e ordenar cada um individualmente, o algoritmo consegue atingir um desempenho muito superior (próximo de O(n+k)). Essa abordagem deixou claro o poder de dividir um problema grande em partes menores dentro de Estrutura de Dados.

A união de POO com Estrutura de Dados ficou visível ao criarmos métodos genéricos em Java. Com o uso de Generics e interfaces, conseguimos fazer com que os mesmos algoritmos de ordenação pudessem ser reutilizados para ordenar diferentes tipos de objetos no sistema, promovendo a reutilização de código e facilitando correções futuras.

4.4 Resumo dos Algoritmos e Desempenho

Para organizar as conclusões obtidas nos testes do projeto, a tabela abaixo resume as principais características de cada algoritmo estudado:

Algoritmo	Melhor Caso	Pior Caso	Uso Recomendado no Sistema
Bubble Sort	O(n)	O(n²)	Vetores bem pequenos ou fins de aprendizado
Binary Insertion Sort	O(n)	O(n²)	Conjuntos médios onde buscas rápidas ajudam

Bucket Sort	O(n + k)	O(n²)	Grandes volumes com dados bem distribuídos

No final das contas, o trabalho mostrou que POO e Estrutura de Dados andam lado a lado. Enquanto a orientação a objetos deixa o código limpo, fácil de entender e bem protegido, as estruturas de dados e os algoritmos garantem que o programa funcione com velocidade e eficiência.

5 - RESULTADOS E DISCUSSÃO

Este trabalho tem como objetivo apresentar um estudo comparativo aprofundado sobre três algoritmos de ordenação: Bubble Sort, Bucket Sort e Binary Insertion Sort. A pesquisa compreende desde a fundamentação teórica sobre métodos de ordenação até a análise do funcionamento de cada algoritmo, com ênfase na avaliação empírica de desempenho. Para os testes experimentais, foram empregados vetores desordenados contendo 1.000, 5.000 e 10.000 elementos numéricos. Os resultados obtidos demonstraram que, a despeito de suas particularidades algorítmicas, todas as implementações alcançaram desempenhos satisfatórios, concluindo a ordenação dos dados em tempos inferiores a um segundo nos cenários testados.

Todas as comparações apresentadas a seguir são médias de 7 testes executados em todos os 3 algoritmos

A máquina usada para os testes, foi um notebook Acer Preto com 8 gigas de memória ram, processador Intel Core I5 i5-13420H e SSD: 512 GB.

Ambos os testes foram realizados nas mesmas condições.

5.1 Comparação Bubble Sort

O Bubble Sort classifica-se como um algoritmo de ordenação por comparação, cujo funcionamento baseia-se em varreduras sucessivas sobre uma estrutura de dados. A cada passagem, o algoritmo compara elementos adjacentes e realiza a troca caso estejam fora da ordem estipulada. Considere-se, como exemplo ilustrativo, o vetor não ordenado formado pelos elementos [10, 8, 13, 2, 1]. Inicialmente, avaliam-se os dois primeiros itens (10 e 8); identificando-se que 10 é superior a 8, suas posições são permutadas, passando o vetor ao estado [8, 10, 13, 2, 1]. O procedimento avança avaliando o segundo e o terceiro elemento, repetindo-se até o encerramento do arranjo. O ciclo é reiterado continuamente até que uma iteração completa transcorra sem a ocorrência de trocas, critério de parada que atesta que o vetor se encontra ordenado (Blog da Trybe , 2022).

A despeito de apresentar uma complexidade assintótica quadrática, expressa pela notação $O(n^2)$, que restringe sua aplicabilidade em cenários de alta demanda computacional, o Bubble Sort detém um valor pedagógico significativo. A simplicidade de sua mecânica operacional e a facilidade de abstração de seus passos tornam-no uma ferramenta didática indispensável para a introdução aos conceitos fundamentais de análise e projeto de algoritmos de ordenação.

No que tange ao consumo de ciclos de processamento (ticks), como ilustrado na tabela 2, o Bubble Sort obteve uma média de 239.783,29 ticks para a ordenação do vetor de 1.000 elementos. Com a ampliação da massa de dados para 5.000 elementos, a média registrada elevou-se para 454.081,86 ticks conforme é mostrado na tabela 3, alcançando 1.289.595,57 ticks no cenário com 10.000 elementos desordenados referenciado pela tabela 4.

Tabela 2 - Comparação Bubble Sort 1.000 números

BUBBLESORT		
N° do Teste	Tempo em ms	N° de Ticks
1	46,3008	463008
2	21,6633	216633
3	19,0372	190372
4	17,6062	176062
5	18,7647	187647
6	20,975	209750
7	23,5011	235011

Fonte: Elaborada pelos autores.

Tabela 3 - Comparação Bubble Sort 5.000 números

BUBBLESORT		
N° do Teste	Tempo em ms	N° de Ticks
1	46,5609	465609
2	49,555	495550
3	45,4209	454209
4	46,4305	464305
5	38,0546	380546
6	47,6675	476675
7	44,1679	441679

Fonte: Elaborada pelos autores.

Tabela 4 - Comparação Bubble Sort 10.000 números

BUBBLESORT		
N° do Teste	Tempo em ms	N° de Ticks
1	122,6111	1226111
2	152,0842	1520842
3	128,276	1282760
4	120,2698	1202698
5	125,1295	1251295
6	128,1338	1281338
7	126,2125	1262125

Fonte: Elaborada pelos autores.

5.2 Comparação Bucket Sort

O Bucket Sort é um algoritmo de ordenação distributiva fundamentado na fragmentação de um conjunto de dados em sub estruturas menores, denominadas baldes (buckets). No procedimento adotado, define-se uma estrutura auxiliar contendo 10 baldes receptores. A etapa inicial compreende o mapeamento dos elementos do vetor primário: para cada valor, calcula-se o índice do balde correspondente por meio da multiplicação pelo número total de baldes, aplicando-se

o truncamento para número inteiro. Concluída a distribuição de toda a massa de dados, os elementos contidos em cada partição são ordenados por meio de um algoritmo auxiliar estável — tipicamente o Insertion Sort. Por fim, realiza-se a etapa de concatenação, na qual os elementos dos baldes ordenados são reinseridos sequencialmente no vetor original, consolidando a ordenação final (Geeksforgeeks, 2025).

Uma característica distintiva do Bucket Sort reside no emprego de um algoritmo secundário para ordenar o conteúdo interno de cada partição, estratégia que viabiliza a redução do esforço computacional e otimiza o tempo de execução global. Nos cenários experimentais avaliados, o método evidenciou elevada eficiência com grandes volumes de dados, superando substancialmente o desempenho obtido pelo Bubble Sort e pelo Binary Insertion Sort.

No que tange ao consumo de ciclos de processamento (ticks), como mostrado na tabela 5 o Bubble Sort obteve uma média de 162.412,57 ticks para a ordenação do vetor de 1.000 elementos. Com a ampliação da massa de dados para 5.000 elementos, conforme a tabela 6 ilustra, a média registrada elevou-se para 189.940,86 ticks, alcançando 237.384,14 ticks no cenário com 10.000 elementos desordenados, disponíveis para visualização da tabela 7.

Tabela 5 - Comparação Bucket Sort 1.000 números

BUCKETSORT		
N° do Teste	Tempo em ms	N° de Ticks
1	11,1241	111241
2	24,2931	242931
3	14,3886	143886
4	13,7255	137255
5	15,3987	153987
6	10,1859	101859
7	24,5729	245729

Fonte: Elaborada pelos autores.

Tabela 6 - Comparação Bucket Sort 5.000 números

BUCKETSORT		
N° do Teste	Tempo em ms	N° de Ticks
1	30,8572	308572
2	14,4307	144307
3	10,5967	105967
4	18,3058	183058
5	17,4872	174872
6	21,4944	214944
7	19,7866	197866

Fonte: Elaborada pelos autores.

Tabela 7 - Comparação Bucket Sort 10.000 números

BUCKETSORT		
N° do Teste	Tempo em ms	N° de Ticks
1	27,1045	271045
2	29,6713	296713
3	21,0851	210851
4	22,4987	224987
5	39,3455	393455
6	12,3086	123086
7	14,1552	141552

Fonte: Elaborada pelos autores.

5.3 Comparação Binary Insertion Sort

O Binary Insertion Sort constitui uma otimização do método clássico de inserção direta, diferenciando-se pelo emprego da busca binária para minimizar a quantidade de comparações necessárias à inserção dos elementos. Estruturalmente, o vetor é particionado de forma lógica em dois segmentos: um subarranjo filtrado à esquerda e um subarranjo não filtrado à direita. Tomando-se o

primeiro item como ordenado por definição, o algoritmo itera do segundo até o último elemento do vetor, armazenando o valor da iteração corrente em uma variável denominada chave. A localização do índice de inserção é determinada por meio de busca binária aplicada estritamente sobre a partição previamente ordenada (intervalo de 0 a i−1). Uma vez identificada a posição adequada, os elementos maiores são deslocados à direita, permitindo que a chave seja alocada em sua posição definitiva (vetor[pos] = chave). (Geeksforgeeks, 2025).

O Binary Insertion Sort evidenciou alta eficiência na manipulação de conjuntos de dados reduzidos; contudo, diante de volumes mais expressivos, demonstrou perda perceptível de desempenho quando confrontado com algoritmos assintoticamente superiores. O método configura-se como uma evolução do Insertion Sort convencional, uma vez que substitui a varredura linear por uma busca binária, minimizando o custo associado às comparações para a determinação do ponto de inserção.

No que tange ao consumo de ciclos de processamento (ticks), na tabela 8 é ilustrado que, o Bubble Sort obteve uma média de 137.337,14 ticks para a ordenação do vetor de 1.000 elementos. Conforme a tabela 9 mostra, houve uma ampliação da massa de dados para 5.000 elementos, a média registrada elevou-se para 213.048,29 ticks, alcançando 447.986,43 ticks no cenário com 10.000 elementos desordenados, conforme mostra a tabela 10.

Tabela 8 - Comparação Binary Insertion Sort 1.000 números

BINARYSORT		
N° do Teste	Tempo em ms	N° de Ticks
1	10,9481	109481
2	15,1044	151044
3	11,3435	113435
4	20,5644	205644
5	9,6373	96373
6	14,6893	146893
7	13,849	138490

Fonte: Elaborada pelos autores.

Tabela 9 - Comparação Binary Insertion Sort 5.000 números

BINARYSORT		
N° do Teste	Tempo em ms	N° de Ticks
1	19,2626	192626
2	14,5875	145875
3	28,0658	280658
4	29,8965	298965
5	24,2012	242012
6	14,4786	144786
7	18,6416	186416

Fonte: Elaborada pelos autores.

Tabela 10 - Comparação Binary Insertion Sort 10.000 números

BINARYSORT		
N° do Teste	Tempo em ms	N° de Ticks
1	44,0824	440824
2	43,4818	434818
3	49,8584	498584
4	38,3246	383246
5	57,6193	576193
6	40,7233	407233
7	39,5007	395007

Fonte: Elaborada pelos autores.

6 - CONSIDERAÇÕES FINAIS

O desenvolvimento das Atividades Práticas Supervisionadas (APS) possibilitou uma análise teórica e experimental de diferentes técnicas de ordenação de dados, permitindo compreender a importância da escolha adequada de algoritmos para a eficiência dos sistemas computacionais. Por meio do estudo comparativo dos algoritmos Bubble Sort, Binary Insertion Sort e Bucket Sort, foi possível identificar suas principais características, vantagens e limitações, relacionando os fundamentos da disciplina de Estruturas de Dados com situações práticas de processamento de informações.

A análise realizada demonstrou que, embora os três algoritmos tenham a mesma finalidade de organizar conjuntos de dados, suas estratégias de funcionamento apresentam diferenças significativas. O Bubble Sort destacou-se pela simplicidade de implementação e facilidade de compreensão, características que justificam sua utilização no ensino dos fundamentos de ordenação. Entretanto, sua complexidade computacional quadrática nos casos médio e pior representa uma limitação para aplicações que envolvem grandes volumes de informações, devido à quantidade de comparações e trocas necessárias durante sua execução.

O Binary Insertion Sort apresentou uma abordagem que busca otimizar o processo de inserção por meio da utilização da busca binária, reduzindo o número de comparações necessárias para identificar a posição adequada de cada elemento. Apesar dessa vantagem, o algoritmo ainda depende do deslocamento dos elementos para realizar as inserções, mantendo a complexidade temporal quadrática nos casos mais desfavoráveis. Essa característica evidencia que a otimização de determinadas operações não implica, necessariamente, uma redução equivalente da complexidade geral do algoritmo.

Por sua vez, o Bucket Sort demonstrou uma estratégia distinta, baseada na distribuição dos elementos em estruturas intermediárias, permitindo a organização dos dados em grupos menores. Sua principal vantagem está na possibilidade de alcançar maior eficiência quando os valores apresentam uma distribuição adequada entre os buckets. Contudo, seu desempenho está condicionado às características

dos dados de entrada, à distribuição dos elementos e ao método utilizado para ordenar cada grupo, fatores que devem ser considerados durante sua aplicação.

Em relação à etapa experimental, os testes realizados com conjuntos de 1.000, 5.000 e 10.000 elementos permitiram observar o comportamento dos algoritmos diante de diferentes volumes de dados. A utilização de condições equivalentes de execução e o cálculo da média de sete testes por cenário contribuíram para a padronização da análise comparativa. Conforme os resultados apresentados, os três algoritmos concluíram as operações de ordenação em tempos inferiores a um segundo nos cenários avaliados. Entretanto, esses resultados devem ser interpretados considerando as condições específicas do ambiente experimental, não sendo suficientes, isoladamente, para determinar a superioridade de um método em todas as situações.

Diante dos aspectos analisados, considera-se que os objetivos estabelecidos para o trabalho foram alcançados, uma vez que o estudo permitiu compreender o funcionamento dos algoritmos selecionados, examinar suas características de complexidade computacional e realizar uma avaliação prática de desempenho. Além disso, a atividade contribuiu para o desenvolvimento de competências relacionadas à implementação de algoritmos, à análise de resultados experimentais e à aplicação dos conhecimentos teóricos na resolução de problemas computacionais.

Conclui-se, portanto, que a eficiência de um algoritmo de ordenação não depende exclusivamente de sua capacidade de organizar corretamente os dados, mas também de fatores como o tamanho do conjunto de entrada, a distribuição dos elementos, a quantidade de operações realizadas e os recursos computacionais disponíveis. Dessa forma, a pesquisa reforça a importância da análise criteriosa das diferentes técnicas de ordenação, demonstrando que a seleção do método mais adequado deve considerar as necessidades e particularidades de cada aplicação. O conhecimento adquirido ao longo do desenvolvimento da APS constitui, assim, uma contribuição relevante para a formação acadêmica em Ciência da Computação e para a compreensão dos princípios envolvidos na construção de soluções computacionais eficientes.

7 - REFERÊNCIAS BIBLIOGRÁFICAS

ALGORITHM ROOM. **Binary Sort** (Binary Insertion Sort). [S. l.]: Algorithm Room, 2026. Disponível em: https://www.algorithmroom.com/dsa/binary-sort-binary-insertion-sort. Acesso em: 22 out. 2026.

BLOG DA TRYBE. **Bubble Sort: o que é e como usar? Exemplos práticos!**. [S. l.]: Trybe, 2022. Disponível em: https://blog.betrybe.com/tecnologia/bubble-sort-tudo-sobre/. Acesso em: 22 out. 2026.

CODDY. **Bubble Sort Visualization**. [S. l.]: Coddy, 2026. Disponível em: https://coddy.tech/visualize/sorting/bubble-sort. Acesso em: 22 out. 2026.

CORMEN, T. H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

CORMEN, T. H. et al. **Algoritmos: Teoria e Prática**. 4. ed. São Paulo, 2024.

CORMEN, Thomas H. et al. **Introduction to algorithms**. 4. ed. Cambridge, Massachusetts: The MIT Press, 2022.

DOUGLAS, Henry. **Algoritmo de ordenação: tipos, complexidade e qual usar em cada caso**. [S. l.]: Shiftmind, 2026. Disponível em: https://shiftmind.com.br/algoritmo-de-ordenacao/. Acesso em: 22 out. 2026.

FATEC FRANCA. **Estudo analítico do desempenho de algoritmos de ordenação**. Revista Edufatec, Franca, v. 2, n. 5, p. 75-90, 2023. Disponível em: https://revistaedufatec.fatecfranca.edu.br/wp-content/uploads/2023/04/edufatec-n05v2a05.pdf. Acesso em: 28 out. 2026.

GEEKSFORGEEKS. **Binary Insertion Sort**. [S. l.]: GeeksforGeeks, 2025. Disponível em: https://www.geeksforgeeks.org/dsa/binary-insertion-sort/. Acesso em: 5 out. 2026.

GEEKSFORGEEKS. **Bucket Sort**. [S. l.]: GeeksforGeeks, 2025. Disponível em: https://www.geeksforgeeks.org/dsa/bucket-sort-2/. Acesso em: 6 out. 2026.

GEEKSFORGEEKS. **Bucket Sort - Python**. [S. l.]: GeeksforGeeks, 2025. Disponível em: https://www.geeksforgeeks.org/dsa/bucket-sort-in-python/. Acesso em: 22 out. 2026.

HARVARD UNIVERSITY. **CS50: Introduction to Computer Science — Week 3: Algorithms**. Cambridge, 2026.

KNUTH, Donald Ervin. **The Art of Computer Programming: Volume 3, Sorting and Searching**. 2. ed. Reading, Massachusetts: Addison-Wesley, 1998.

KRIVILEV, A. V.; GAVRICHENKO, E. O. **Analysis of the binary insertion sort algorithm and its modification based on comparison forests**. Programming and

Computer Software, v. 52, n. 1, p. 40–59, 2026. Disponível em: https://doi.org/10.7868/S3034584726010033. Acesso em: 6 out. 2026.

PROGRAMIZ. **Insertion Sort** (With Code in Python/C++/Java/C). [S. l.]: Programiz, [s. d.]. Disponível em: https://www.programiz.com/dsa/insertion-sort. Acesso em: 22 out. 2026.

SEDGEWICK, Robert; FLAJOLET, Philippe. **An Introduction to the Analysis of Algorithms**. 2. ed. Boston: Addison-Wesley, 2013.

SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4. ed. Boston: Addison-Wesley, 2011.

SIMIC, Milos. **Binary Insertion Sort**. [S. l.]: Baeldung, 2024. Disponível em: https://www.baeldung.com/cs/binary-insertion-sort. Acesso em: 5 out. 2026.

TECNODIGITAL. **El Bucketsort: Ordenar Datos Rápidamente**. [S. l.]: InformaTec Digital, 2025. Disponível em: https://informatecdigital.com/el-bucketsort/. Acesso em: 22 out. 2026.

TENENBAUM, Aaron M. et al. **Estruturas de dados usando C**. São Paulo: Makron Books, 1995.

THORNTON, Michael. ICS 46 Spring 2022: **Notes and Examples — Comparison-Based Sorting**. Irvine: University of California, 2022. Disponível em:

https://ics.uci.edu/~thornton/ics46/Notes/ComparisonBasedSorting/. Acesso em: 22 out. 2026.

UNIVERSIDADE PAULISTA (UNIP). **Plano de ensino / Manual de Atividades Práticas Supervisionadas (APS): Curso de Ciência da Computação**. São Paulo, 2026.

VIGNATTI, André. **Bucket Sort**. [S. l.]: Universidade Federal do Paraná, [s. d.]. Disponível em: https://www.inf.ufpr.br/vignatti/v/cursos/ci1165/s20.pdf. Acesso em: 6 out. 2026.

WIKILIVROS. **Algoritmos e Estruturas de Dados: Algoritmos de Ordenação**. [S. l.]: Wikilivros, [s. d.]. Disponível em: https://pt.wikibooks.org/wiki/Algoritmos_e_Estruturas_de_Dados/Algoritmos_de_Ordena%C3%A7%C3%A3o. Acesso em: 22 out. 2026.

WIKIPÉDIA. **Bucket sort**. [S. l.]: Wikimedia Foundation, 2021. Disponível em: https://pt.wikipedia.org/wiki/Bucket_sort. Acesso em: 6 out. 2026.

WIKIPÉDIA. **Clube de karatê do Zachary**. [S. l.]: Wikimedia Foundation, 2022. Disponível em: https://pt.wikipedia.org/wiki/Clube_de_karat%C3%AA_do_Zachary. Acesso em: 22 out. 2026.

WIRTH, Niklaus. **Algorithms and Data Structures**. Englewood Cliffs: Prentice-Hall, 1985.

ZIVIANI, N. **Projeto de algoritmos: com implementações em Pascal e C**. 2. ed. rev. e ampl. São Paulo: Pioneira Thomson Learning, 2004.

APÊNDICE

Código fonte do Binary Insertion Sort:

```
//Chamando biblioteca que tem a função printf
#include <stdio.h>

//Criando nossa função de busca binaria, passando como parametro o vetor, o item a ser comparado, o valor do menor e maior indice do vetor
int buscaBinaria(int vetor[], int item, int menorIndice, int maiorIndice) {
   //Criando um looping while, que permanecerá enquanto meu menor indice, for menor ou igual ao meu maior indice
  while(menorIndice <= maiorIndice) {
     //Inicializamos a variavel de indice medio igual ao meu menor indice somado com a diferença do meu maior indice com o menor indice, por fim divididos por 2
    int indiceMedio = menorIndice + (maiorIndice - menorIndice) / 2;

      //Verifica se o meu valor onde o indice medio se encontra e igual ao do item comparado
    if (vetor[indiceMedio] == item) {
       return indiceMedio + 1; //Se for, adiciona meu valor de indice medio uma casa a direita
    }

     //Verifica se o meu valor onde o indice medio se encontra e menor ao do item comparado
    if (vetor[indiceMedio] < item) {
      menorIndice = indiceMedio + 1; //Se for, adiciona meu valor de indice menor uma casa a direita do meu indice medio
    }
      //Verifica se o meu valor onde o indice medio se encontra e maior ao do item comparado
    else {
       maiorIndice = indiceMedio - 1; //Se for, adiciona meu valor de indice maior uma casa a esquerda do meu indice medio
```

```
    }
  }

  //Caso menor indice, maior que meu maior indice. Retorne o menor indice
  return menorIndice;
}

//Criando o método de inserção por binarios, passando como parametro o vetor e o tamanho do mesmo
void binaryInsertionSort(int vetor[], int tamanho) {
  //Percorrendo o vetor por completo
  for (int indice1 = 1; indice1 < tamanho; indice1++) {
    //Inicializamos a variavel chave, sendo igual ao meu valor do vetor onde o indice é igual ao indice1 do nosso laço
    int chave = vetor[indice1];
      //Inicializamos a variavel pos, que guardara a posição retornada pela busca binaria
    int pos = buscaBinaria(vetor, chave, 0, indice1 - 1);

    //Inicializamos o indice2, que é igual ao indice1 - 1
    int indice2 = indice1 - 1;
    //Criamos um looping while, enquanto o indice2 for maior ou igual a pos da busca binaria
    while (indice2 >= pos) {
      vetor[indice2 + 1] = vetor[indice2]; //Se for define que o valor de posição do vetor onde o indice for o indice 2 - 1, será movido uma casa a esquerda
      indice2--; //Subtrai 1 do indice2
    }

    //Depois de terminar o looping, define que o valor do vetor onde o indice e igual ao indice2 - 1, será nossa nova chave
    vetor[indice2 + 1] = chave;
  }
}
```

```
//Aqui criamos nosso main e chamamos nosso método
int main() {
  //Inicialimos o vetor com 7 números, com indices de 0 a 6
  int vetor[] = {};
   //Inicializamos a variavel tamanho, que será igual ao total de bytes do vetor por completo, dividido pelo tamanho de bytes de um elemento do vetor. Nesse caso: tamanho = 28 / 4 = 7
  int tamanho = sizeof(vetor) / sizeof(vetor[0]);

  //Chamando nosso metodo, passando os parametros vetor e tamanho
  binaryInsertionSort(vetor, tamanho);

  //Imprimindo o nosso vetor ordenado
  printf("Vetor ordenado: ");
  //Percorre todo o vetor, imprimindo um número ao lado do outro
  for (int indice = 0; indice < tamanho; indice++) {
    printf("%d ", vetor[indice]);
  }
  //Quebra de linha
  printf("\n");

  //Retorna 0 se tudo der certo
  return 0;
}
```

Código fonte Bucket Sort:

```
#include <stdio.h>
#include <stdlib.h>

/* Estrutura para os nós da lista encadeada (Baldes) */
struct No {
    float valor;
```

```
    struct No *proximo;
};

/* Função para inserir um elemento de forma ordenada no balde */
struct No *inserirOrdenado(struct No *lista, float valor) {
    struct No *novoNo = (struct No *)malloc(sizeof(struct No));
    novoNo->valor = valor;
    novoNo->proximo = NULL;

    // Se a lista estiver vazia ou o novo valor for menor que o primeiro
    if (lista == NULL || lista->valor >= valor) {
        novoNo->proximo = lista;
        return novoNo;
    }

    // Localiza o ponto de inserção correto
    struct No *atual = lista;
    while (atual->proximo != NULL && atual->proximo->valor < valor) {
        atual = atual->proximo;
    }

    novoNo->proximo = atual->proximo;
    atual->proximo = novoNo;

    return lista;
}

/* Função principal do Bucket Sort (Adaptada para qualquer intervalo de números) */
void bucketSort(float vetor[], int n) {
    if (n <= 0) return;

    // 1. Encontra o menor e o maior valor do vetor para fazer a normalização
    float menor = vetor[0];
    float maior = vetor[0];
```

```
for (int i = 1; i < n; i++) {
    if (vetor[i] < menor) menor = vetor[i];
    if (vetor[i] > maior) maior = vetor[i];
}

// Se todos os elementos forem iguais, já está ordenado
if (menor == maior) return;

// 2. Inicializa os baldes (vetor de ponteiros para nós)
struct No **baldes = (struct No **)calloc(n, sizeof(struct No *));
if (baldes == NULL) {
    printf("Erro de alocação de memória.\n");
    return;
}

// 3. Distribuição (Scatter) com mapeamento dinâmico para qualquer intervalo
for (int i = 0; i < n; i++) {
    // Mapeia o valor linearmente para o intervalo [0, 1) e multiplica por n
    int indiceBalde = (int)((vetor[i] - menor) / (maior - menor) * (n - 1));

    // Insere o elemento ordenadamente no respetivo balde
    baldes[indiceBalde] = inserirOrdenado(baldes[indiceBalde], vetor[i]);
}

// 4. Coleta (Gather): junta todos os baldes de volta no vetor original
int indiceVetor = 0;
for (int i = 0; i < n; i++) {
    struct No *atual = baldes[i];
    while (atual != NULL) {
        vetor[indiceVetor++] = atual->valor;

        // Libera a memória do nó após o uso
        struct No *temp = atual;
        atual = atual->proximo;
```

```
            free(temp);
        }
    }

    // Libera o vetor de ponteiros principal
    free(baldes);
}

// Função auxiliar para imprimir o vetor
void imprimirVetor(float vetor[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%.2f ", vetor[i]);
    }
    printf("\n");
}

int main() {
      // Testando com números fora do intervalo de 0 a 1 (ex: negativos e números maiores)
      float vetor[] = {1511, -18463, -37407, 63983, -10575, -91889, 64628, -77710, 15482, 22387};
    int n = sizeof(vetor) / sizeof(vetor[0]);

    printf("Vetor original: \n");
    imprimirVetor(vetor, n);

    bucketSort(vetor, n);

    printf("\nVetor ordenado: \n");
    imprimirVetor(vetor, n);

    return 0;
}
```

Código Fonte do Bubble Sort:

```
//Chamando biblioteca que tem a função printf
#include <stdio.h>

//Criando algoritmo de ordenação Bubble sort, com o vetor e o tamanho como parametro
void bubbleSort(int vetor[], int tamanho) {
  //Percorremos e verificamos todos os indicies, o numero do vetor de vezes, para ter certeza que verificamos todos os numeros
    for (int indice1 = 0; indice1 < tamanho - 1; indice1++) {
        int trocou = 0; //Reinicia a variavel troca
         //Percorremos e verificamos todos os indicies, o numero do vetor de vezes - o indicie 1 (para evitarmos verificar o ultimo numero, pois o mesmo já foi comparado com todos os outros, sendo declarado o maior). Para ter certeza que verificamos todos os numeros
        for (int indice2 = 0; indice2 < tamanho - 1 - indice1; indice2++) {
          //Verifica se o vetor do indicie 2 é maior que o proximo indicie comparado
            if (vetor[indice2] > vetor[indice2 + 1]) {
              //Inicia uma variavel que guardará o valor da variavel maior de indicie 2
                int guardaValor = vetor[indice2];
                //Troca a variavel antiga pela menor, fazendo assim a troca de posições
                vetor[indice2] = vetor[indice2 + 1];
                //Na posição antiga onde ficava o numero menor, e passado o valor que foi guardado anteriormente, assim os ordenando em ordem crescente
                vetor[indice2 + 1] = guardaValor;
                    //Por fim, definimos o atributo troca igual a 1, para dizermos que houve uma troca
                trocou = 1;
            }
        }
        //Verifica se trocou é falso, ou seja valor = 0
        if (!trocou) {
            break; //Quebra o laço for, pois não há mais trocas
```

```
        }
    }
}
//Aqui chamamos nosso algoritmo
int main() {
  //Criando um vetor de 7 números com indicies de 0 a 6
     int vetor[] = {1511, -18463, -37407, 63983, -10575, -91889, 64628, -77710, 15482,
-64598};
      //Criando a variavel de tamanho, com uma divisão entre a quantidade total de
bytes do vetor pela quantidade total de um elemento do vetor
    int tamanho = sizeof(vetor) / sizeof(vetor[0]);

    //Chamando nosso algoritmo
    bubbleSort(vetor, tamanho);

    //Imprimindo o nosso vetor ordenado
    printf("Vetor ordenado: ");
    //Percorre todo o vetor, imprimindo um número ao lado do outro
    for (int indice1 = 0; indice1 < tamanho; indice1++) {
        printf("%d ", vetor[indice1]);
    }

    //Retorna 0 se tudo der certo
    return 0;
}
```